beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in beamer documents. Named overlay specifications are very handy both during edition and to manage complex and variable beamer overlay specifications. In particular, they allow to replace raw numbers in beamer <...> overlay specifications by logical identifiers. Demonstration files are available for download as part of the development repository. As a side effect, this is a solution to this latex.org forum query.

Contents

1 Installation

1.1 Package manager

When not already available, beanoves package may be installed using a TEX distribution's package manager, either from the graphical user interface, or with the relevant command (`tlmgr` for TEX Live and `mpm` for MiKTEX). This should install files beanoves.sty and its debug version beanoves-debug.sty as well as beanoves-doc.pdf documentation.

1.2 Manual installation

The beanoves source files are available from the source repository. They can also be fetched from the CTAN repository.

1.3 Usage

The beanoves package is imported by putting `\RequirePackage{beanoves}` in the preamble of a LATEX document that uses the beamer class. Should the package cause problems, its features can be temporarily deactivated with simple commands `\BeanovesOff` and `\BeanovesOn`.

2 Minimal example

The LaTeX document below is a contrived example to show how the `beamer` overlay specifications have been extended. More demonstration files are available from the beanoves source repository.

```
1  \documentclass{beamer}
2  \RequirePackage{beanoves}
3  \begin{document}
4  \Beanoves {
5    A = 1:4,
6    B = A.last::3,
7    C = B.next,
8  }
9  \begin{frame}
10   {\Large Frame \insertframenumber}
11   {\Large Slide \insertslidenumber}
12 -- \visible<?(A.1)> {Only on slide 1}\\
13 -- \visible<?(B.range)> {Only on slides 4 to 6}\\
14 -- \visible<?(C.1)> {Only on slide 7}\\
15 -- \visible<?(A.2)> {Only on slide 2}\\
16 -- \visible<?(B.2:B.last)> {Only on slides 5 to 6}\\
17 -- \visible<?(C.2)> {Only on slide 8}\\
18 -- \visible<?(A.next)-> {From slide 5}\\
19 -- \visible<?(B.3:B.last)> {Only on slide 6}\\
20 -- \visible<?(C.3)> {Only on slide 9}\\
21 \end{frame}
22 \end{document}
```

On line 4, we use the `\Beanoves` command to declare *named overlay sets*. On line 5, we declare an overlay set named 'A', which is a range starting at slide 1 and ending at slide 4. On line 12, the extended *named overlay specification* `?(A.1)` stands for 1 because 1 is the first index of the overlay set named A. On line 15, `?(A.2)` stands for 2 whereas on line 18, `?(A.next)` stands for 5. On line 6, we declare a second overlay set named 'B', starting after the 3 slides of 'A' namely 4. Its length is 3 meaning that its last slide number is 6, thus each `?(B.last)` is replaced by 6. The next slide number after slide range 'B' is 7 which is also the start of the third slide range due to line 7.

3 Named overlay sets

3.1 Presentation

Within a beamer frame, there are different slides that appear in turn according to overlay specifications. The main overlay set is a range of integers covering all the slide numbers, from one to the total amount of slides. In general, an overlay set is a range of positive integers identified by a unique name. The main practical interest is that such sets may be defined relative to one another, we can even have lists of overlay sets. Finally, we can use these lists to build and organize beamer overlay specifications logically.

3.2 Named overlay reference

`A.1`, `C.2` are *named overlay references*, as well as `A` and `X!A.B.C.2`. More precisely, they are string identifiers, each one referencing a well defined static integer or range to be used in beamer overlay specifications. They have 3 components:

1. the *frame identifier* ⟨*id*⟩ before the exclamation mark `!`, like `X`, in `X!A.B.C.2`, optional
2. the *path* ⟨c_1⟩.....⟨c_j⟩ like `A.B.C`, required ($j > 0$).
3. the *attributes* .⟨$attr_1$⟩.⟨...⟩.⟨$attr_k$⟩ like `.1`, `.2`, `.reset.first`...

The *full path* is ⟨*id*⟩`!`⟨c_1⟩.....⟨c_j⟩. The *frame ids* and path components ⟨*c*⟩'s are alphanumerical case sensitive identifiers, with possible underscores but with no space. Unicode symbols above `U+00A0` are allowed if the underlying TEX engine supports it. Only the *frame id* is allowed to be empty, in which case it may apply to any common frame. The *path components* must not consist of only lowcase letters[1]. The attributes ⟨*attr*⟩'s may be signed integers of one of the keywords defined below.

The mapping from *named overlay references* to sets of integers is defined at the global TEX level to allow its use in `\begin{frame}<...>` and to share the same overlay sets between different frames. Hence the *frame id* due to the need to possibly target a particular frame.

3.3 Defining named overlay sets

In order to define *named overlay sets*, we can either execute the next `\Beanoves` command before a beamer frame environment, or use the custom `beanoves` option of this environment.

`\Beanoves` `\Beanoves*`

`\Beanoves{`⟨ref_1⟩`=`⟨$spec_1$⟩`,...,`⟨ref_j⟩`=`⟨$spec_j$⟩`}`

Each ⟨*ref*⟩ key is a *named overlay reference* whereas each ⟨*spec*⟩ is an *overlay set specifier*. When the same ⟨*ref*⟩ key is used multiple times, only the last one is taken into account.

When performed at the document level, the `\Beanoves` command starts by cleaning what was set by previous calls. When performed inside LATEX environments, each new call cumulates with the previous one. Notice that the argument of this function can contain macros: they will be exhaustively expanded at resolution time[2].

`beanoves`

`beanoves = {`⟨ref_1⟩`=`⟨$spec_1$⟩`,...,`⟨ref_j⟩`=`⟨$spec_j$⟩`}`

The `\Beanoves` arguments take precedence over both the `\Beanoves*` arguments and the `beanoves` options. This allows to provide an overlay name only when not already defined, which is helpfull when the very same frame source is included multiple times in different contexts. Notice that ⟨*ref*⟩`=1` can be shortened to ⟨*ref*⟩.

3.3.1 Value specifiers

Hereafter ⟨*value*⟩ denotes a numerical expression.

[1] This will avoid collisions with the `fp` module of `expl3`.
[2] Precision is needed about the exact time when the expansion occurs.

Standalone

⟨*ref*⟩=⟨*value*⟩, a *value specifier* for a single number. When =⟨*value*⟩ is omitted it defaults to =1.

The numerical expressions are evaluated and then rounded using `\fp_eval:n`. They can contain mathematical functions and *named overlay references* defined above. If this reference concerns a set of values, only the first one is considered.

The corresponding overlay set can be seen as a *value counter*.

3.3.2 Colon delimited range specifiers

In beamer, integer ranges are denoted by ⟨*first*⟩-⟨*last*⟩. In order to authorize algebraic expressions for ⟨*first*⟩ and ⟨*last*⟩, the dash character - is replaced by a colon : in beanoves. Hereafter ⟨*first*⟩, ⟨*last*⟩ and ⟨*length*⟩ are placeholders for *value specifiers*.

Standalone

⟨*ref*⟩=⟨*first*⟩:,
⟨*ref*⟩=⟨*first*⟩::, for the infinite range of signed integers starting at and including ⟨*first*⟩.

⟨*ref*⟩=⟨*first*⟩:⟨*last*⟩,
⟨*ref*⟩=⟨*first*⟩::⟨*length*⟩,
⟨*ref*⟩=:⟨*last*⟩::⟨*length*⟩,
⟨*ref*⟩=::⟨*length*⟩:⟨*last*⟩, are variants for the same finite range of signed integers starting at and including ⟨*first*⟩, ending at and including ⟨*last*⟩, provided ⟨*first*⟩ + ⟨*length*⟩ = ⟨*last*⟩ + 1. ⟨*first*⟩ can be omitted, in which case it defaults to 1. Additionally :⟨*last*⟩ and ::⟨*length*⟩ are then equivalent.

⟨*ref*⟩=:⟨*last*⟩,
⟨*ref*⟩=::⟨*length*⟩, are syntactic sugar when ⟨*first*⟩ is 1.

3.3.3 List specifiers

⟨*ref*⟩=[⟨def_1⟩,...,⟨def_j⟩], where ⟨def_k⟩, $1 \leq k \leq j$, is one of

- ⟨*index*⟩=⟨*value*⟩,
- ⟨*value*⟩, a shortcut for ⟨*i*⟩=⟨*value*⟩, ⟨*i*⟩ being the smallest positive integer such that ⟨*ref*⟩.⟨*i*⟩ is not already defined.

The first step is to remove previous ⟨*ref*⟩ related definitions, then execute the various ⟨*ref*⟩.⟨*i*⟩=⟨*value*⟩ definitions in the order given. Here is the implementation.

⟨*ref*⟩={⟨def_1⟩,...,⟨def_j⟩}, where ⟨def_k⟩, $1 \leq k \leq j$, is one of

- ⟨*name*⟩=⟨*spec*⟩,
- ⟨*name*⟩ for ⟨*name*⟩=1,

The first step is to remove previous ⟨*ref*⟩ related definitions, then execute the various ⟨*ref*⟩.⟨*name*⟩=⟨*spec*⟩ definitions in the order given. In a final step, the ⟨*name*⟩'s are collected in a comma sepated list to initialize ⟨*ref*⟩ with. ⟨*spec*⟩ is any specifier. Here is the implementation.

4 Resolution of `?(...)` query expressions

This is the key feature of the beanoves package, extending beamer *overlay specifications* normally included between pointed brackets. Before the *overlay specifications* are processed by the beamer class, the beanoves package scans them for any occurrence of `?(`⟨*queries*⟩`)`. Each one is then evaluated and replaced by its resolved static counterpart. The overall result is finally forwarded to the beamer class.

The ⟨*queries*⟩ argument is a comma separated list of individual ⟨*query*⟩'s processed from left to right as explained below. Notice that nesting a `?(`⟨`...`⟩`)` query expression inside another query expression is supported, embedded expressions being resolved first.

The named overlay sets defined above are queried for integer numerical values that will be passed to beamer. Turning an *overlay query* into the static expression it represents, as when above `?(A.1)` was replaced by `1`, is denoted by *overlay query resolution* or simply *resolution*. The process starts by replacing any *query reference* by its value as explained below until obtaining numerical expressions that are evaluated and finally rounded to the nearest integer to feed beamer with either ranges or numbers. When the *query reference* is a previously declared ⟨*ref*⟩, like `A` after `A=1`, it is simply replaced by the corresponding declared ⟨*value*⟩, here `1`. Otherwise, we use *implicit overlay queries* and their *resolution rules* depending on the definition of the named overlay set. Hereafter ⟨*i*⟩ denotes a signed integer whereas ⟨*value*⟩, ⟨*first*⟩, ⟨*last*⟩, ⟨*length*⟩ and ⟨*i expr*⟩ stand for raw integers or more general numerical expressions that are evaluated beforehands.

Resolution occurs only when requested and the result is cached for performance reason.

4.1 Range overlay queries

We give queries after named overlay sets definitions.

⟨*ref*⟩ `=` ⟨*first*⟩`:` as well as ⟨*first*⟩`::` define a range limited from below:

overlay query	resolution
⟨*ref*⟩	⟨*first*⟩`-`
⟨*ref*⟩`.1`	⟨*first*⟩
⟨*ref*⟩`.2`	⟨*first*⟩ + 1
⟨*ref*⟩`.`⟨*i*⟩	⟨*first*⟩ + ⟨*i*⟩ − 1
⟨*ref*⟩`[`⟨*i expr*⟩`]`	⟨*first*⟩ + ⟨*i expr*⟩ − 1
⟨*ref*⟩`.previous`	⟨*first*⟩ − 1
⟨*ref*⟩`.first`	⟨*first*⟩

Notice that ⟨*ref*⟩`.previous` and ⟨*ref*⟩`.0` are most of the time synonyms.

⟨*ref*⟩ `=` ⟨*first*⟩`:`⟨*last*⟩ as well as variants ⟨*first*⟩`::`⟨*length*⟩, `::`⟨*length*⟩`:`⟨*last*⟩ or `:`⟨*last*⟩`::`⟨*length*⟩, which are equivalent provided ⟨*first*⟩+⟨*length*⟩ = ⟨*last*⟩+1.

Define a range limited from both above and below:

overlay query	resolution
⟨*ref*⟩	⟨*first*⟩-⟨*last*⟩
⟨*ref*⟩.1	⟨*first*⟩
⟨*ref*⟩.2	⟨*first*⟩ + 1
⟨*ref*⟩.⟨*i*⟩	⟨*first*⟩ + ⟨*i*⟩ − 1
⟨*ref*⟩[⟨*i expr*⟩]	⟨*first*⟩ + ⟨*i expr*⟩ − 1
⟨*ref*⟩.previous	⟨*first*⟩ − 1
⟨*ref*⟩.first	⟨*first*⟩
⟨*ref*⟩.last	⟨*last*⟩
⟨*ref*⟩.next	⟨*last*⟩ + 1
⟨*ref*⟩.length	⟨*length*⟩

Notice that the resolution of the ⟨*ref*⟩ overlay query is a beamer range and not an algebraic difference, negative integers do not make sense there while in beamer context.

In the frame example below, we use the \BeanovesResolve command for the demonstration. It is mainly used for debugging and testing purposes.

```
\Beanoves {
  A = 3:8, % or similarly A = 3::6, A = ::6:8 and A = :8::6
}
\begin{frame} {Frame \insertframenumber} {Slide \insertslidenumber}
\ttfamily
\BeanovesResolve[show](A)          == 3-8,
\BeanovesResolve[show](A.1)        == 3,
\BeanovesResolve[show](A.-1)       == 1,
\BeanovesResolve[show](A.previous) == 2,
\BeanovesResolve[show](A.first)    == 3,
\BeanovesResolve[show](A.last)     == 8,
\BeanovesResolve[show](A.next)     == 9,
\BeanovesResolve[show](A.length)   == 6,
\end{frame}
```

⟨*ref*⟩ = [...]
⟨*ref*⟩ = {...} See the list range specifiers in section 3.3.3.

4.2 Value counter queries

⟨*ref*⟩ = ⟨*value*⟩ defines a counter value.

overlay query	resolution
⟨*ref*⟩	⟨*value*⟩
⟨*ref*⟩.1	⟨*value*⟩
⟨*ref*⟩.2	⟨*value*⟩ + 1
⟨*ref*⟩.⟨*i*⟩	⟨*value*⟩ + ⟨*i*⟩ − 1
⟨*ref*⟩[⟨*i expr*⟩]	⟨*value*⟩ + ⟨*i expr*⟩ − 1
⟨*ref*⟩.previous	⟨*value*⟩ − 1
⟨*ref*⟩.first	⟨*value*⟩
⟨*ref*⟩.last	⟨*value*⟩
⟨*ref*⟩.next	⟨*value*⟩ + 1

Additionnaly, resolution rules are provided for dedicated *overlay queries*. Here, ⟨*ref*⟩ is considered a standard programming variable:

⟨*ref*⟩=⟨*set expression*⟩, resolve ⟨*set expression*⟩, assign the result to the ⟨*ref*⟩ and use it. It defines ⟨*ref*⟩ globally if not already done. Here ⟨*set expression*⟩ is the longest character sequence with no space[3]. To remove this limitation, use an embedded query expression or enclose the `clist` item within a pair of braces.

⟨*ref*⟩+=⟨*integer expression*⟩, resolve ⟨*integer expression*⟩ into ⟨*integer*⟩, advance ⟨*ref*⟩ by ⟨*integer*⟩ and use the result.

++⟨*ref*⟩, increment ⟨*ref*⟩ by 1 and use it.

⟨*ref*⟩++, use ⟨*ref*⟩ and then increment it by 1.

IN PROGRESS

This can be used for an indirection.

```
\Beanoves {
  A = 1,
  B = [ 10 = 100 ],
  C = 10,
}
\begin{frame} {Frame \insertframenumber} {Slide \insertslidenumber}
\ttfamily
\BeanovesResolve[show](A.C)      == \BeanovesResolve[show](A.10) == 10,
\BeanovesResolve[show](B.C)      == \BeanovesResolve[show](B.10) == 100,
\BeanovesResolve[show](A[C+=10]) == \BeanovesResolve[show](A.20) == 20,
\BeanovesResolve[show](A.C)      == \BeanovesResolve[show](A.20) == 20,
\BeanovesResolve[show](A.C+=10)  == \BeanovesResolve[show](A.20) == 20+10,
\BeanovesResolve[show](A.C)      == \BeanovesResolve[show](A.20) == 30,
\BeanovesResolve[show](B.C)      == \BeanovesResolve[show](B.20) == 20,
\end{frame}
```

In order to decrement a counter, one can increment with a negative value, no dedicated syntax is provided yet.

For each new frame, these counters are reset to the value they were initialized with. Sometimes, resetting the counter manually is necessary, for example when managing tikz overlay material.

\BeanovesReset `\BeanovesReset [`⟨*options*⟩`] {`⟨*ref*$_1$⟩`[=`⟨*spec*$_1$⟩`],...,` ⟨*ref*$_j$⟩`[=`⟨*spec*$_j$⟩`]}`

This command is very similar to `\Beanoves`, except that a standalone ⟨*ref*$_i$⟩ resets the counter to the last value it was initialized with, and that it is meant to be used inside a `frame` environment. Each `=`⟨*spec*$_j$⟩ is optional and defaults to `=1`. When the `all` option is provided, some internals that were cached for performance reasons are cleared as well.

4.3 The `beamer` counters

While inside a `frame` environment, it is possible to save the current value of the `beamerpauses` counter that controls whether elements should appear on the current slide. For that, we can execute one of `\Beanoves{`⟨*ref*⟩`=pauses}` or in a query

[3] The parser for algebraic expression is very rudimentary.

`?(...(⟨ref⟩=pauses)...)`. Then later on, we can use `?(...⟨ref⟩...)` to refer to this saved value in the same frame[4]. Next frame source is an example of usage.

```
1 \begin{frame}
2 \visible<+->{A}\\
3 \visible<+->{B\Beanoves{afterB=pauses}}\\
4 \visible<+->{C}\\
5 \visible<?(afterB)>{other C}\\
6 \visible<?(afterB.previous)>{other B}\\
7 \end{frame}
```

"A" first appears on slide 1, "B" on slide 2 and "C" on slide 3. On line 2, `afterB` takes the value of the `beamerpauses` counter once updated, *id est* 3. "B" and "other B" as well as "C" and "other C" appear at the same time. If the `beamerpauses` counter is not suitable, we can execute instead one of `\Beanoves{⟨ref⟩=slideinframe}` or inside a query `?(⟨...⟩(⟨ref⟩=slideinframe)⟨...⟩)`. It uses the numerical value of `\insertslideinframe`.

4.4 Multiple queries

It is possible to replace the comma separated list of queries `?(⟨query`$_1$`⟩),...,?(⟨query`$_j$`⟩)` with the shorter single query `?(⟨query`$_1$`⟩,...,⟨query`$_j$`⟩)`.

4.5 Frame id

Except for very special situations, the *frame ids* can be left unspecified. When no *frame id* was explicitly provided, beanoves uses the *last frame id* and, if the resolution fails, an empty *frame id*. At the beginning of each frame, the *last frame id* is set to the *frame id* of the current frame, which is denoted *current frame id* and is empty by default. Then it gets updated after each named reference resolution where a *frame id* is explicitly given. For example, the first time `A.1` reference is resolved within a given frame, it is first translated to `⟨last frame id⟩!A.1`, but when used just after `Y!C.2`, for example, it becomes a shortcut to `Y!A.1` because the *last frame id* is then `Y`.

In order to set the *frame id* of the current frame to frame `⟨id⟩`, use the new `beanoves id` option of the beamer frame environment.

`beanoves id` `beanoves id=⟨frame id⟩,`

We can use the same `⟨frame id⟩` for different frames to share named overlay sets. Another possibility offered by the beanoves package to share named overlay sets is a fall back mechanism, for example when `X!A` cannot be resolved, `!A` is resolved instead.

[4] See stackexchange for an alternative that needs at least two passes.

4.6 Resolution command

\BeanovesResolve `\BeanovesResolve [⟨setup⟩] {⟨queries⟩}`

This function resolves the ⟨*queries*⟩, which are like the argument of `?(⟨...⟩)` instructions: a comma separated list of single ⟨*query*⟩'s. It is mainly used for debugging purposes. The optional ⟨*setup*⟩ is a key–value list:

`show` the result is left into the input stream

`in:N=`⟨*command*⟩ the result is stored into ⟨*command*⟩.

5 Support

See the source repository. One can report issues there.

6 Implementation

Identify the internal prefix (LaTeX3 DocStrip convention).

```
1 ⟨@@=bnvs⟩
```

6.1 Package declarations

```
2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

6.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over expl.

6.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over LaTeX3. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where LaTeX3 would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, bnvs will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentionned below. For a better reading experience, "`ref`" will generally stand for `\l__bnvs_ref_tl`, whereas

"path sequence" will generally stand for \l__bnvs_path_seq. Other similar shortcuts are used as well.

Functions with BNVS in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the beanoves package.

\BNVS:c	\BNVS:c {⟨*cs core name*⟩}
\BNVS_l:cn	\BNVS_l:cn {⟨*local variable core name*⟩} {⟨*type*⟩}
\BNVS_g:cn	\BNVS_g:cn {⟨*global variable core name*⟩} {⟨*type*⟩}

These are naming functions internally used to focus on the discrimining part of variable or function names.

```
\cs_new:Npn \BNVS:c    #1    {  __bnvs_#1    }
\cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
\cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }
```

\BNVS_use_raw:N	\BNVS_use_raw:N ⟨*cs*⟩
\BNVS_use_raw:c	\BNVS_use_raw:c {⟨*cs name*⟩}
\BNVS_use_raw:Nc	\BNVS_use_raw:Nc ⟨*function*⟩ {⟨*cs name*⟩}
\BNVS_use_raw:nc	\BNVS_use_raw:nc {⟨*tokens*⟩} {⟨*cs name*⟩}
\BNVS_use:c	\BNVS_use:c {⟨*cs core*⟩}
\BNVS_use:Nc	\BNVS_use:Nc ⟨*function*⟩ {⟨*cs core*⟩}
\BNVS_use:nc	\BNVS_use:nc {⟨*tokens*⟩} {⟨*cs core*⟩}

\BNVS_use_raw:c is a convenient wrapper over \use:c. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like \BNVS_use:c. The other are used to expand \use:c enough before usage by ⟨*function*⟩ or ⟨*tokens*⟩. The first argument of ⟨*function*⟩ has type N. The next token after ⟨*tokens*⟩ will have type N too. ⟨*cs name*⟩ is a full cs name whereas ⟨*cs core*⟩ will be prepended with the appropriate prefix specific to the beanoves package.

```
\cs_new:Npn \BNVS_use_raw:N #1 { #1 }
\cs_new:Npn \BNVS_use_raw:c #1 {
  \exp_last_unbraced:No
  \BNVS_use_raw:N { \cs:w #1 \cs_end: }
}

\cs_new:Npn \BNVS_use:c #1 {
  \BNVS_use_raw:c { \BNVS:c { #1 } }
}

\cs_new:Npn \BNVS_use_raw:NN #1 #2 {
  #1 #2
}

\cs_new:Npn \BNVS_use_raw:nN #1 #2 {
  #1 #2
}

\cs_new:Npn \BNVS_use_raw:Nc #1 #2 {
  \exp_args:NNc \BNVS_use_raw:NN #1 { #2 }
}
```

```
\cs_new:Npn \BNVS_use_raw:nc #1 #2 {
  \exp_last_unbraced:Nno
  \BNVS_use_raw:nN { #1 } { \cs:w #2 \cs_end: }
}

\cs_new:Npn \BNVS_use:Nc #1 #2 {
  \BNVS_use_raw:Nc #1 { \BNVS:c { #2 } }
}

\cs_new:Npn \BNVS_use:nc #1 #2 {
  \BNVS_use_raw:nc { #1 } { \BNVS:c { #2 } }
}

\tl_new:N \l__bnvs_last_unbraced_tl
\cs_new:Npn \BNVS_tl_last_unbraced:nv #1 {
  \tl_set:Nn \l__bnvs_last_unbraced_tl { #1 }
  \BNVS_tl_use:nc { \exp_last_unbraced:NV \l__bnvs_last_unbraced_tl }
}
\cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
  \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
}
\cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
  \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
}

\cs_new:Npn \BNVS_log:n #1 { }
\cs_generate_variant:Nn \BNVS_log:n { x }
```

6.3.1 Debug

\BNVS_DEBUG_ON:
\BNVS_DEBUG_OFF:
\BNVS_DEBUG_push:nn
\BNVS_DEBUG_pop:

\BNVS_DEBUG_ON:
\BNVS_DEBUG_OFF:
\BNVS_DEBUG_push:nn {⟨*types on*⟩} {⟨*types off*⟩}
\BNVS_DEBUG_pop:

These functions activate or deactivate the debug mode. They are only active in the beanoves-debug package. Manage debug messaging for one given ⟨*type*⟩ or ⟨*types*⟩. The implementation is not publicly exposed. The ⟨*type*⟩ is a single letter or **.

6.3.2 Facility layer: Functions

\BNVS_new:cpn
\BNVS_set:cpn

\BNVS_new:cpn is like \cs_new:cpn except that the name argument is tagged for beanoves package. Similarly for \BNVS_set:cpn.

```
\cs_new:Npn \BNVS_new:cpn #1 {
  \cs_new:cpn { \BNVS:c { #1 } }
}

\cs_new:Npn \BNVS_set:cpn #1 {
  \cs_set:cpn { \BNVS:c { #1 } }
}
```

```
\cs_generate_variant:Nn \cs_generate_variant:Nn { c }
\cs_new:Npn \BNVS_generate_variant:cn #1 {
  \cs_generate_variant:cn { \BNVS:c { #1 } }
}
```

6.3.3 logging

\BNVS_warning:n
\BNVS_warning:x
\BNVS_error:n
\BNVS_error:x
\BNVS_fatal:n
\BNVS_fatal:x

\BNVS_warning:n {⟨*message*⟩}
\BNVS_error:n {⟨*message*⟩}
\BNVS_fatal:n {⟨*message*⟩}

Very rudimentary. No long message available for error recovery.

```
\msg_new:nnn { beanoves } { :n } { #1 }
\msg_new:nnn { beanoves } { :nn } { #1~(#2) }

\cs_new:Npn \BNVS_warning:n {
  \msg_warning:nnn { beanoves } { :n }
}
\cs_new:Npn \BNVS_warning:x {
  \msg_warning:nnx { beanoves } { :n }
}

\cs_new:Npn \BNVS_error:n {
  \msg_error:nnn { beanoves } { :n }
}
\cs_generate_variant:Nn \BNVS_error:n { x }

\cs_new:Npn \BNVS_fatal:n {
  \msg_fatal:nnn { beanoves } { :n }
}
\cs_new:Npn \BNVS_fatal:x {
  \msg_fatal:nnx { beanoves } { :n }
}
\cs_new:Npn \BNVS_fatal_unreachable: {
  \BNVS_fatal:n { Unreachable }
}
\cs_new:Npn \BNVS_fatal_unreachable: { }
```

6.3.4 Facility layer: Variables

\BNVS_N_new:c
\BNVS_v_new:c

\BNVS_N_new:c {⟨*type*⟩}
\BNVS_v_new:c {⟨*type*⟩}

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. ⟨*type*⟩ is one of `tl`, `seq`...

```
\cs_new:Npn \BNVS_N_new:c #1 {
```

```
  \cs_new:cpn { BNVS_#1:c } ##1 {
    l \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
  }
  \cs_new:cpn { BNVS_#1_new:c } ##1 {
    \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
  }
  \cs_new:cpn { BNVS_#1_use:c } ##1 {
    \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
  }
  \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
    \BNVS_use_raw:Nc
      ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
  }
  \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
    \BNVS_use_raw:nc
      { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
  }
}
\cs_new:Npn \BNVS_v_new:c #1 {
  \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
    \BNVS_use_raw:nc
      { \exp_args:NV ##1 }
      { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
  }
  \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
    \BNVS_use_raw:nc
      { \exp_args:NnV \BNVS_use:c { ##1 } }
      { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
  }
  \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
    \BNVS_use_raw:nc
      { \exp_args:NnV \use:n { ##1 } }
      { \BNVS_use_raw:c { BNVS_#1:c } { ##2 } }
  }
}
\BNVS_N_new:c { bool }
\BNVS_N_new:c { int }
\BNVS_v_new:c { int }
\BNVS_N_new:c { tl }
\BNVS_v_new:c { tl }
\cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
  \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
}
\cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
  \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
}
\cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
  \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
}
\BNVS_N_new:c { str }
\BNVS_v_new:c { str }
```

```
\BNVS_N_new:c { seq }
\BNVS_v_new:c { seq }
\cs_undefine:N \BNVS_N_new:c
```

\BNVS_use:Ncn `\BNVS_use:Ncn` ⟨*function*⟩ {⟨*core name*⟩} {⟨*type*⟩}

```
\cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
  \BNVS_use_raw:c { BNVS_#3_use:Nc }   #1   { #2 }
}
\cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
  \BNVS_use_raw:c { BNVS_#3_use:nc } { #1 } { #2 }
}
\cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
  \BNVS_use_raw:c { BNVS_#3_use:Nv }   #1   { #2 }
}
\cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
  \BNVS_use_raw:c { BNVS_#3_use:nv } { #1 } { #2 }
}
\cs_new:Npn \BNVS_use:Ncncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:Ncn   #1   { #2 } { #3 }
  }
}
\cs_new:Npn \BNVS_use:ncncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:ncn { #1 } { #2 } { #3 }
  }
}
\cs_new:Npn \BNVS_use:Nvncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:Nvn   #1   { #2 } { #3 }
  }
}
\cs_new:Npn \BNVS_use:nvncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:nvn { #1 } { #2 } { #3 }
  }
}
\cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
  \BNVS_use:ncn {
    \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
  }
}
\cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
  \BNVS_use:ncn {
    \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
  }
}
```

\BNVS_new_c:cn \BNVS_new_c:nc {⟨*type*⟩} {⟨*core name*⟩}

```
\cs_new:Npn \BNVS_new_c:nc #1 #2 {
  \BNVS_new:cpn { #1_#2:c } {
    \BNVS_use_raw:c { BNVS_#1_use:nc } { \BNVS_use_raw:c { #1_#2:N } }
  }
}

\cs_new:Npn \BNVS_new_cn:nc #1 #2 {
  \BNVS_new:cpn { #1_#2:cn } ##1 {
    \BNVS_use:ncn { \BNVS_use_raw:c { #1_#2:Nn } } { ##1 } { #1 }
  }
}

\cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
  \BNVS_new:cpn { #2:cnn } ##1 {
    \BNVS_use:Ncn { #3 } { ##1 } { #1 }
  }
}

\cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_cnn:ncN { #1 } { #1_#2 }
  } { #1_#2:Nnn }
}

\cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
  \BNVS_new:cpn { #2:cnv } ##1 ##2 {
    \BNVS_tl_use:nv {
      \BNVS_use:Ncn #3 { ##1 } { #1 } {  ##2 }
    }
  }
}

\cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_cnv:ncN { #1 } { #1_#2 }
  } { #1_#2:Nnn }
}

\cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
  \BNVS_new:cpn { #2:cnx } ##1 ##2 {
    \exp_args:Nnx \use:n {
      \BNVS_use:Ncn #3 { ##1 } { #1 } {  ##2 }
    }
  }
}

\cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_cnx:ncN { #1 } { #1_#2 }
  } { #1_#2:Nnn }
}
```

```
\cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
  \BNVS_new:cpn { #2:cc } ##1 ##2 {
    \BNVS_use:Ncncn #3 { ##1 } { #1 } { ##2 } { #4 }
  }
}

\cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_cc:ncNn { #1 } { #1_#2 }
  } { #1_#2:NN }
}

\cs_new:Npn \BNVS_new_cc:nc #1 #2 {
  \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
}

\cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
  \BNVS_new:cpn { #2:cn } ##1 {
    \BNVS_use:Ncn #3 { ##1 } { #1 }
  }
}

\cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_cn:ncNn { #1 } { #1_#2 }
  } { #1_#2:Nn }
}

\cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
  \BNVS_new:cpn { #2:cv } ##1 ##2 {
    \BNVS_use:nvn {
      \BNVS_use:Ncn #3 { ##1 } { #1 }
    } { ##2 } { #4 }
  }
}

\cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_cv:ncNn { #1 } { #1_#2 }
  } { #1_#2:Nn }
}

\cs_new:Npn \BNVS_new_cv:nc #1 #2 {
  \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
}

\cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
  \BNVS_use_raw:Nc   #1   { \BNVS_l:cn { #2 } { #3 } }
}

\cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
  \BNVS_use_raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
}
```

```
\cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
  \BNVS_use_raw:Nc   #1   { \BNVS_g:cn { #2 } { #3 } }
}

\cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
  \BNVS_use_raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
}
```

\BNVS_new_conditional:cpnn `\BNVS_new_conditional:cpnn {⟨core⟩} ⟨parameter⟩ {⟨conditions⟩} {⟨code⟩}`

```
\cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

\cs_new:Npn \BNVS_new_conditional:cpnn #1 {
  \prg_new_conditional:cpnn { \BNVS:c { #1 } }
}
\cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
\cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
  \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
}

\cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
    \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vn:cNnn { #1 }
  } { #1:nn TF } { #2 }
}

\cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
    \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vc:cNnn { #1 }
  } { #1:ncTF }
}
```

```
\cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
  \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
    \BNVS_use:nvn {
      \BNVS_use:Nvn #2 { ##1 } { #3 }
    } { ##2 } { #4 } { ##3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvc:cNnnn { #1 }
  } { #1:nncTF }
}

\cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
    \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vc:cNn { #1 }
  } { #1:ncTF }
}

\cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
    \BNVS_tl_use:nv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } { ##3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvc:cNn { #1 }
  } { #1:nncTF }
}
```

6.3.5 Regex

```
\cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
  \BNVS_use_raw:Nc #1 { c \BNVS:c { #2 } _regex }
}
```

6.3.6 Token lists

__bnvs_tl_clear:c	__bnvs_tl_clear:c {⟨*core key tl*⟩}
__bnvs_tl_use:c	__bnvs_tl_use:c {⟨*core*⟩}
__bnvs_tl_set_eq:cc	__bnvs_tl_count:c {⟨*core*⟩}
__bnvs_tl_set:cn	__bnvs_tl_set_eq:cc {⟨*lhs core name*⟩} {⟨*rhs core name*⟩}
__bnvs_tl_set:(cv|cx|ce)	__bnvs_tl_set:cn {⟨*core*⟩} {⟨*tl*⟩}
__bnvs_tl_put_left:cn	__bnvs_tl_set:cv {⟨*core*⟩} {⟨*value core name*⟩}
__bnvs_tl_put_right:cn	__bnvs_tl_put_left:cn {⟨*core*⟩} {⟨*tl*⟩}
__bnvs_tl_put_right:(cx|ce|cv)	__bnvs_tl_put_right:cn {⟨*core*⟩} {⟨*tl*⟩}
	__bnvs_tl_put_right:cv {⟨*core*⟩} {⟨*value core name*⟩}

These are shortcuts to

- \tl_clear:c {l__bnvs_⟨*core*⟩_tl}
- \tl_use:c {l__bnvs_⟨*core*⟩_tl}
- \tl_set_eq:cc {l__bnvs_⟨*lhs core*⟩_tl}{l__bnvs_⟨*rhs core*⟩_tl}
- \tl_set:cv {l__bnvs_⟨*core*⟩_tl}{l__bnvs_⟨*value core*⟩_tl}
- \tl_set:cx {l__bnvs_⟨*core*⟩_tl}{⟨*tl*⟩}
- \tl_put_left:cn {l__bnvs_⟨*core*⟩_tl}{⟨*tl*⟩}
- \tl_put_right:cn {l__bnvs_⟨*core*⟩_tl}{⟨*tl*⟩}
- \tl_put_right:cv {l__bnvs_⟨*core*⟩_tl}{l__bnvs_⟨*value core*⟩_tl}

\BNVS_new_conditional_vnc:cn	\BNVS_new_conditional_vnc:cn {⟨*core*⟩} {⟨*conditions*⟩}

Creates ⟨*core*⟩:vncTF from ⟨*core*⟩:nncTF.

```
\cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
    \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

```
\cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vnc:cNn { #1 }
  } { #1:nncTF }
}
```

\BNVS_new_conditional_vvnc:cn \BNVS_new_conditional_vvnc:cn {⟨*core*⟩} {⟨*conditions*⟩}

Creates ⟨*core*⟩:vvncTF from ⟨*core*⟩:nnncTF.

```
\cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
    \BNVS_tl_use:nv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } { ##3 } { ##4 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvnc:cNn { #1 }
  } { #1:nnncTF }
}

\cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
    \BNVS_tl_use:nvv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } { ##3 } { ##4 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

\BNVS_new_conditional_vvvc:cn \BNVS_new_conditional_vvvc:cn {⟨*core*⟩} {⟨*conditions*⟩}

Creates ⟨*core*⟩:vvvcTF from ⟨*core*⟩:nnncTF.

```
\cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvvc:cNn { #1 }
  } { #1:nnncTF }
}
```

```
\cs_new:Npn \BNVS_new_tl_c:c {
  \BNVS_new_c:nc { tl }
}
\BNVS_new_tl_c:c { clear }
\BNVS_new_tl_c:c { use }
\BNVS_new_tl_c:c { count }

\BNVS_new:cpn { tl_set_eq:cc } #1 #2 {
  \BNVS_use:ncncn { \tl_set_eq:NN } { #1 } { tl } { #2 } { tl }
}

\cs_new:Npn \BNVS_new_tl_cn:c {
  \BNVS_new_cn:nc { tl }
}

\cs_new:Npn \BNVS_new_tl_cv:c #1 {
  \BNVS_new_cv:ncn { tl } { #1 } { tl }
}
\BNVS_new_tl_cn:c { set }
\BNVS_new_tl_cv:c { set }

\BNVS_new:cpn { tl_set:cx } {
  \exp_args:Nnx \__bnvs_tl_set:cn
}

\BNVS_new:cpn { tl_set:ce } {
  \exp_args:Nne \__bnvs_tl_set:cn
}
\BNVS_new_tl_cn:c { put_right }
\BNVS_new_tl_cv:c { put_right }
% \BNVS_generate_variant:cn { tl_put_right:cn } { cx }

\BNVS_new:cpn { tl_put_right:cx } {
  \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
}

\BNVS_new:cpn { tl_put_right:ce } {
  \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
}
\BNVS_new_tl_cn:c { put_left }
\BNVS_new_tl_cv:c { put_left }
% \BNVS_generate_variant:cn { tl_put_left:cn } { cx }

\BNVS_new:cpn { tl_put_left:cx } {
  \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
}
```

`\__bnvs_tl_if_empty:c`*TF* `\__bnvs_tl_if_empty:cTF` {⟨*core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
`\__bnvs_tl_if_blank:v`*TF* `\__bnvs_tl_if_blank:vTF` {⟨*core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
`\__bnvs_tl_if_eq:cn`*TF* `\__bnvs_tl_if_eq:cnTF` {⟨*core*⟩} {⟨*tl*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

These are shortcuts to

- `\tl_if_empty:cTF {l__bnvs_`⟨*core*⟩`_tl}` {⟨*yes:*⟩} {⟨*no:*⟩}
- `\tl_if_eq:cnTF {l__bnvs_`⟨*core*⟩`_tl}`{⟨*tl*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

```
\cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
    \BNVS_use:Ncn #3 { ##1 } { #1 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
  } { #1_#2:NTF }
}
\BNVS_new_conditional_c:ncn { tl } { if_empty } { p, T, F, TF }

\BNVS_new_conditional:cpnn { tl_if_blank:v } #1 { T, F, TF } {
  \BNVS_tl_use:Nv \tl_if_blank:nTF { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}

\cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
    \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
  } { #1_#2:NnTF }
}
\BNVS_new_conditional_cn:ncn { tl } { if_eq } { T, F, TF }

\cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:cv } ##1 ##2 { #4 } {
    \BNVS_use:nvn {
      \BNVS_use:Ncn #3 { ##1 } { #1 }
    } { ##2 } { #1 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

```
\cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
  } { #1_#2:NnTF }
}
\BNVS_new_conditional_cv:ncn { tl } { if_eq } { T, F, TF }

\BNVS_new:cpn { gput_right:cccn } #1 #2 #3 {
  \tl_gput_right:cn { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
}
```

6.3.7 Strings

__bnvs_str_if_eq:vv*TF* `\__bnvs_str_if_eq:vvTF` {⟨*core*⟩} {⟨*tl*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

These are shortcuts to

- `\str_if_eq:ccTF {l__bnvs_`⟨*core*⟩`_tl}`{⟨*yes:*⟩} {⟨*no:*⟩}

```
\cs_new:Npn \BNVS_new_conditional_vv:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vv } ##1 ##2 { #3 } {
    \BNVS_tl_use:nv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvnc:cNn { #1 }
  } { #1:nnTF }
}

\cs_new:Npn \BNVS_new_conditional_vn:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:vn } ##1 ##2 { #4 } {
    \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_conditional_vn:ncNn { #1 } { #1_#2 }
  } { #1_#2:nnTF }
}
\BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }
```

```
\cs_new:Npn \BNVS_new_conditional_vv:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
    \BNVS_use:nvn {
      \BNVS_use:Nvn #3 { ##1 } { #1 }
    }  { ##2 } { #1 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
  \BNVS_use_raw:nc {
    \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
  } { #1_#2:nnTF }
}
\BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }
```

6.3.8 Sequences

`\__bnvs_seq_count:c`	`\__bnvs_seq_new:c {`⟨*core*⟩`}`
`\__bnvs_seq_clear:c`	`\__bnvs_seq_count:c {`⟨*core*⟩`}`
`\__bnvs_seq_set_eq:cc`	`\__bnvs_seq_clear:c {`⟨*core*⟩`}`
`\__bnvs_seq_gset_eq:cc`	`\__bnvs_seq_set_eq:cc {`⟨$core_1$⟩`} {`⟨$core_2$⟩`}`
`\__bnvs_seq_use:cn`	`\__bnvs_seq_use:cn {`⟨*core*⟩`} {`⟨*separator*⟩`}`
`\__bnvs_seq_item:cn`	`\__bnvs_seq_item:cn {`⟨*core*⟩`} {`⟨*integer expression*⟩`}`
`\__bnvs_seq_remove_all:cn`	`\__bnvs_seq_remove_all:cn {`⟨*core*⟩`} {`⟨*tl*⟩`}`
`\__bnvs_seq_put_left:cv`	`\__bnvs_seq_put_right:cn {`⟨*seq core*⟩`} {`⟨*tl*⟩`}`
`\__bnvs_seq_put_right:cn`	`\__bnvs_seq_put_right:cv {`⟨*seq core*⟩`} {`⟨*tl core*⟩`}`
`\__bnvs_seq_put_right:cv`	`\__bnvs_seq_set_split:cnn {`⟨*seq core*⟩`} {`⟨*tl*⟩`} {`⟨*separator*⟩`}`
`\__bnvs_seq_set_split:cnn`	`\__bnvs_seq_pop_left:cc {`⟨$core_1$⟩`} {`⟨$core_2$⟩`}`
`\__bnvs_seq_set_split:(cnv|cnx)`	
`\__bnvs_seq_pop_left:cc`	

These are shortcuts to

- `\seq_set_eq:cc {l__bnvs_`⟨$core_1$⟩`_seq} {l__bnvs_`⟨$core_2$⟩`_seq}`
- `\seq_count:c {l__bnvs_`⟨*core*⟩`_seq}`
- `\seq_use:cn {l__bnvs_`⟨*core*⟩`_seq} {`⟨*separator*⟩`}`
- `\seq_item:cn {l__bnvs_`⟨*core*⟩`_seq} {`⟨*integer expression*⟩`}`
- `\seq_remove_all:cn {l__bnvs_`⟨*core*⟩`_seq} {`⟨*tl*⟩`}`
- `\seq_clear:c {l__bnvs_`⟨*core*⟩`_seq}`
- `\seq_put_right:cv {l__bnvs_`⟨*core*⟩`_seq} {l__bnvs_`⟨*core*⟩`_tl}`
- `\seq_set_split:cnn {l__bnvs_`⟨*core*⟩`_seq} {l__bnvs_`⟨*core*⟩`_tl}\KWNmeta{tl}`

```
\BNVS_new_c:nc   { seq } { count }
\BNVS_new_c:nc   { seq } { clear }
\BNVS_new_cn:nc  { seq } { use }
\BNVS_new_cn:nc  { seq } { item }
\BNVS_new_cn:nc  { seq } { remove_all }
\BNVS_new_cn:nc  { seq } { map_inline }
\BNVS_new_cc:nc  { seq } { set_eq }
\BNVS_new_cc:nc  { seq } { gset_eq }
\BNVS_new_cv:ncn { seq } { put_left  } { tl }
\BNVS_new_cn:ncn { seq } { put_right } { tl }
\BNVS_new_cv:ncn { seq } { put_right } { tl }
\BNVS_new_cnn:nc { seq } { set_split }
\BNVS_new_cnv:nc { seq } { set_split }
\BNVS_new_cnx:nc { seq } { set_split }
\BNVS_new_cc:ncn { seq } { pop_left  } { tl }
\BNVS_new_cc:ncn { seq } { pop_right } { tl }
```

__bnvs_seq_if_empty:c*TF* \ __bnvs_seq_if_empty:cTF {⟨*seq core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_seq_get_right:cc*TF* \ __bnvs_seq_get_right:ccTF {⟨*seq core*⟩} {⟨*tl core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_seq_pop_left:cc*TF*
__bnvs_seq_pop_right:cc*TF*

```
\cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
    \BNVS_use:ncncn {
      \BNVS_use_raw:c { #1_#2:NNTF }
    } { ##1 } { #1 } { ##2 } { #3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
\BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
\BNVS_new_conditional_cc:ncnn
  { seq } { get_right } { tl } { T, F, TF }
\BNVS_new_conditional_cc:ncnn
  { seq } { pop_left  } { tl } { T, F, TF }
\BNVS_new_conditional_cc:ncnn
  { seq } { pop_right } { tl } { T, F, TF }
```

6.3.9 Integers

__bnvs_int_new:c
__bnvs_int_use:c
__bnvs_int_zero:c
__bnvs_int_inc:c
__bnvs_int_decr:c
__bnvs_int_set:cn
__bnvs_int_set:cv

```
\__bnvs_int_new:c  {⟨core⟩}
\__bnvs_int_use:c  {⟨core⟩}
\__bnvs_int_incr:c {⟨core⟩}
\__bnvs_int_decr:c {⟨core⟩}
\__bnvs_int_set:cn {⟨core⟩} {⟨value⟩}
```

These are shortcuts to

- `\int_new:c  {l__bnvs_⟨core⟩_int}`
- `\int_use:c  {l__bnvs_⟨core⟩_int}`
- `\int_incr:c {l__bnvs_⟨core⟩_int}`
- `\int_decr:c {l__bnvs_⟨core⟩_int}`
- `\int_set:cn {l__bnvs_⟨core⟩_int} ⟨value⟩`

```
\BNVS_new_c:nc   { int } { new  }
\BNVS_new_c:nc   { int } { use  }
\BNVS_new_c:nc   { int } { zero }
\BNVS_new_c:nc   { int } { incr }
\BNVS_new_c:nc   { int } { decr }
\BNVS_new_cn:nc  { int } { set  }
\BNVS_new_cv:ncn { int } { set  } { int }
```

6.4 Debug facilities

Typesetting file `beanoves.dtx` creates both beanoves and beanoves-debug style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

6.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a TEX group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the TEX groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

\l___bnvs_I_tl: A frame identifier

\l___bnvs_J_tl: The last frame identifier

\l___bnvs_K_tl: `!` as regex catched group, if non empty.

\l___bnvs_S_tl: A short name, a regex catched group.

\l___bnvs__P__tl: A path, a regex catched group.

\l___bnvs__N__tl: The representation of an unsigned decimal integer.

```
\tl_new:N \l__bnvs_J_tl
\tl_new:N \l__bnvs_I_tl
\tl_new:N \l__bnvs_K_tl
\tl_new:N \l__bnvs_S_tl
\tl_new:N \l__bnvs_P_tl
\tl_new:N \l__bnvs_R_tl
\tl_new:N \l__bnvs_D_tl
\tl_new:N \l__bnvs_B_tl
\tl_new:N \l__bnvs_N_tl
\tl_new:N \l__bnvs_T_tl
\tl_new:N \l__bnvs_U_tl
\tl_new:N \l__bnvs_V_tl
\tl_new:N \l__bnvs_A_tl
\tl_new:N \l__bnvs_C_tl
\tl_new:N \l__bnvs_F_tl
\tl_new:N \l__bnvs_L_tl
\tl_new:N \l__bnvs_Z_tl
\tl_new:N \l__bnvs_Q_tl
\tl_new:N \l__bnvs_a_tl
\tl_new:N \l__bnvs_z_tl
\tl_new:N \l__bnvs_l_tl
\tl_new:N \l__bnvs_r_tl
\tl_new:N \l__bnvs_n_tl
\tl_new:N \l__bnvs_ref_tl
\tl_new:N \l__bnvs_v_tl
\tl_new:N \l__bnvs_b_tl
\tl_new:N \l__bnvs_c_tl
\tl_new:N \l__bnvs_ans_tl
\tl_new:N \l__bnvs_base_tl
\tl_new:N \l__bnvsroup_tl
\tl_new:N \l__bnvs_scan_tl
\tl_new:N \l__bnvs_query_tl
\tl_new:N \l__bnvs_n_incr_tl
\tl_new:N \l__bnvs_incr_tl
\tl_new:N \l__bnvs_plus_tl
\tl_new:N \l__bnvs_rhs_tl
\tl_new:N \l__bnvs_post_tl
\tl_new:N \l__bnvs_suffix_tl
\tl_new:N \l__bnvs_index_tl
\int_new:N \g__bnvs_call_int
\int_new:N \l__bnvs_i_int
\seq_new:N \g__bnvs_def_seq
\seq_new:N \l__bnvs_a_seq
\seq_new:N \l__bnvs_b_seq
\seq_new:N \l__bnvs_ans_seq
\seq_new:N \l__bnvs_Q_seq
\seq_new:N \l__bnvs_R_seq
\seq_new:N \l__bnvs_match_seq
\seq_new:N \l__bnvs_split_seq
\seq_new:N \l__bnvs_P_seq
```

```
\bool_new:N \l__bnvs_parse_bool
\bool_set_false:N \l__bnvs_parse_bool
\bool_new:N \l__bnvs_deep_bool
\bool_set_false:N \l__bnvs_deep_bool
\bool_new:N \l__bnvs_no_range_bool
\bool_set_false:N \l__bnvs_no_range_bool

\BNVS_new:cpn { tl_clear_ans: } {
  \__bnvs_tl_clear:c { ans }
}

\cs_new:Npn \BNVS_error_ans:x {
  \__bnvs_tl_put_right:cn { ans } 0
  \BNVS_error:x
}
```

In order to implement the provide feature, we add getters and setters

```
\BNVS_new:cpn { set_true:c } #1 {
  \exp_args:Nc \bool_set_true:N { \BNVS_l:cn { #1 } { bool } }
}

\BNVS_new:cpn { set_false:c } #1 {
  \exp_args:Nc \bool_set_false:N { \BNVS_l:cn { #1 } { bool } }
}
```

6.6 Infinite loop management

Unending recursivity is managed here.

\g__bnvs_call_int

Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(*End of definition for* `\g__bnvs_call_int`.)

```
\int_const:Nn \c__bnvs_max_call_int { 8192 }
```

__bnvs_greset_call: `\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```
\BNVS_new:cpn  { greset_call: } {
  \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
}
```

__bnvs_if_call:*TF* `\__bnvs_call_do:TF` {⟨*yes:*⟩} {⟨*no:*⟩}

Decrement the `\g__bnvs_call_int` counter globally and execute ⟨*yes:*⟩ if we have not reached 0, ⟨*no:*⟩ otherwise.

```
\BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
  \int_gdecr:N \g__bnvs_call_int
  \int_compare:nNnTF \g__bnvs_call_int > 0 {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

6.7 Overlay specification

6.7.1 Registration

We keep track of the ⟨*id*⟩`!`⟨*path*⟩ combinations and provide looping mechanisms. Hereafter, ⟨*id*⟩, ⟨*path*⟩ and ⟨*key*⟩ allways represent pure strings.

`\__bnvs_g_IPK:w` `\__bnvs_g_IPK:w {⟨id⟩} {⟨path⟩} {⟨key⟩}`
`\__bnvs_g_IP:w` `\__bnvs_g_IP:w {⟨id⟩} {⟨path⟩}`
`\__bnvs_g_I:w` `\__bnvs_g_I:w {⟨id⟩}`

Create a unique global name from the arguments.

```
\BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_tl }
\BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
\BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }
```

`\g__bnvs_I_seq` Global list of globally registered identifiers.

(*End of definition for* `\g__bnvs_I_seq`.)

```
\seq_new:N \g__bnvs_I_seq
```

6.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the ⟨*id*⟩ corresponds to the drive, the ⟨*path*⟩ points to a directory, each directory may contain individual files. File named "`=`" contains initial data whereas file named "`@`" contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named "⟨*N*⟩" and "`@`⟨*N*⟩" are used similarly, ⟨*N*⟩ denoting a normalized integer (no leading `0`, no leading `+` sign). The dot (`.`) is used to separate components of ⟨*path*⟩.

6.7.3 Low level IPK model functions

These are `gset` and `gunset` functions. Each `gset` function must be balanced by a `gunset` function to prevent "memory leaks".

`\__bnvs_gset:nnnn` `\__bnvs_gset:nnnn {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨def⟩}`
`\__bnvs_gset:(nnnv|nnne|vnnn|nvnn|nvnv|vvnn|vvnv)`

Manage the storage. Keep track of all the ⟨*path*⟩'s for a given ⟨*id*⟩ and all the ⟨*key*⟩'s for a given ⟨*id*⟩`!`⟨*path*⟩ combination.

Implementation details, next macros are defined lazily at the global level:

- `\__bnvs_gset@`⟨*id*⟩`!`⟨*path*⟩`:nn {`⟨*key*⟩`} {`⟨*def*⟩`}`,
- `\__bnvs_gset@`⟨*id*⟩`:nnn {`⟨*path*⟩`} {`⟨*key*⟩`} {`⟨*def*⟩`}`,
- `\g__bnvs@`⟨*id*⟩`!_paths_tl`, records all ⟨*path*⟩ for ⟨*id*⟩,
- `\g__bnvs@`⟨*id*⟩`!`⟨*path*⟩`_keys_tl`, records all ⟨*key*⟩ for ⟨*id*⟩`!`⟨*path*⟩,
- `\g__bnvs@`⟨*id*⟩`!`⟨*path*⟩`_keys_tl`,

- \g__bnvs@⟨*id*⟩!⟨*path*⟩/⟨*key*⟩_tl, record the ⟨*def*⟩ for the ⟨*id*⟩!⟨*path*⟩/⟨*key*⟩ combination.

```
\BNVS_new:cpn { gset:Nn } #1 {
  \tl_gclear_new:N #1
  \tl_gset:Nn #1
}
\BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
  \seq_gclear_new:N #1
  \cs_new:Npn #2 ##1 ##2 {
    \seq_if_in:NnF #1 { ##1 } {
      \seq_gput_right:Nn #1 { ##1 }
    }
    \exp_args:Nc \__bnvs_gset:Nn {
      \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
    } { ##2 }
  }
  #2
}
\BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
  \cs_if_exist_use:NF #1 {
    \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
    \seq_gclear_new:N #2
    \cs_new:Npn #1 ##1 {
      \seq_if_in:NnF #2 { ##1 } {
        \seq_gput_right:Nn #2 { ##1 }
      }
      \exp_args:Ncc \__bnvs_gset_IPKV:NNw
      { \__bnvs_g_IP:w { #3 } { ##1 } }
      { __bnvs_gset@#3!##1:nn } { #3 } { ##1 }
    }
    #1
  }
  { #4 }
}
\BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
  \cs_if_exist_use:cF { __bnvs_gset@#1!#2:nn } {
    \exp_args:Ncc \__bnvs_gset:NNnnnn
      { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
  } { #3 } { #4 }
}
\BNVS_new:cpn { gset:vnnn } {
  \BNVS_tl_use:cv { gset:nnnn }
}
\BNVS_new:cpn { gset:nvnn } #1 {
  \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
}
\BNVS_new:cpn { gset:vvnn } {
  \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
}
\BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
  \BNVS_tl_use:nv {
    \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
  }
```

```
}
\BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
  \BNVS_tl_use:nv {
    \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
  }
}
\BNVS_new:cpn { gset:nvnv } {
  \BNVS_tl_use:Nv \__bnvs_gset:nnnv
}
\cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }
```

__bnvs_gset:nnv `\__bnvs_gset:nnv {⟨id⟩} {⟨path⟩} {⟨key⟩}`

Syntactic sugar for `\__bnvs_gset:nnnv {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨key⟩}`.

```
\BNVS_new:cpn { gset:nnv } #1 #2 #3 {
  \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
}
```

__bnvs_gunset:nnn `\__bnvs_gunset:nnv {⟨id⟩} {⟨path⟩} {⟨key⟩}`
__bnvs_gunset:vvn

Removes the specifications for the ⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩ combination.

```
\seq_new:N \l__bnvs_I_seq
\BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
  \seq_clear:N \l__bnvs_I_seq
  \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
    \tl_if_eq:nnTF { ##1 } { #2 } {
      \cs_set:Npn \BNVS_seq_gremove_Nn:n ####1 {
        \seq_put_right:Nn \l__bnvs_I_seq { ####1 }
      }
    } {
      \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
    }
  }
  \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
  \seq_gset_eq:NN #1 \l__bnvs_I_seq
}
\BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
  \__bnvs_seq_gremove_once:Nn #1 { #3 }
  \seq_if_empty:NT #1 {
    \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
    \cs_undefine:c { __bnvs_gset@#2:nnn}
  }
}
\BNVS_new:cpn { gunset:NNnnn } #1 #2 #3 #4 #5 {
  \tl_if_exist:NT #1 {
    \cs_undefine:N #1
    \__bnvs_seq_gremove_once:Nn #2 { #5 }
    \seq_if_empty:NT #2 {
      \exp_args:Nc \__bnvs_gunset_IP:Nw { \__bnvs_g_I:w { #3 } } { #3 } { #4 }
      \cs_undefine:c { __bnvs_gset@#3!#4:nn}
    }
  }
}
\BNVS_new:cpn { gunset:nnn } #1 #2 #3 {
```

```
  \exp_args:Ncc
  \__bnvs_gunset:NNnnn
    { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
    { \__bnvs_g_IP:w  { #1 } { #2 } }
    { #1 } { #2 } { #3 }
}
\BNVS_new:cpn { gunset:vvn } {
  \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
}
```

`\__bnvs_is_gset:nnnTF`	`\__bnvs_is_gset:nnnTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨yes:⟩} {⟨no:⟩}`
`\__bnvs_is_gset:(nvn|vvn|vxn)TF`	`\__bnvs_is_gset:nnTF {⟨id⟩} {⟨path⟩} {⟨yes:⟩} {⟨no:⟩}`
`\__bnvs_is_gset:nnTF`	

Test for the existence of some data for that ⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩ combination. The version with no ⟨*key*⟩ corresponds to key '`=`'.

```
\BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
  \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
    \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { is_gset:vxn } #1 #2 #3 { T, F, TF } {
  \exp_args:NNnx \BNVS_tl_use:Nv
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { is_gset:vvn } #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { is_gset:nn } #1 #2 { T, F, TF } {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
}
```

6.7.4 Loops

`\__bnvs_foreach_I:N`	`\__bnvs_foreach_I:N ⟨function:n⟩`
`\__bnvs_foreach_I:n`	`\__bnvs_foreach_I:n {⟨:n code⟩}`
`\__bnvs_foreach_I_break:`	`\__bnvs_foreach_I_break:`
`\__bnvs_foreach_I_break:n`	`\__bnvs_foreach_I_break:n {⟨code⟩}`

Execute the ⟨*function:n*⟩ or the ⟨*:n code*⟩ for each declared identifier. The break commands work like `\seq_map_break:` and

```
\BNVS_new:cpn { foreach_I:N } {
  \seq_map_function:NN \g__bnvs_I_seq
}
\BNVS_new:cpn { foreach_I:n } {
  \seq_map_inline:Nn \g__bnvs_I_seq
}
```

```
\cs_set_eq:NN \__bnvs_foreach_I_break:  \seq_map_break:
\cs_set_eq:NN \__bnvs_foreach_I_break:n \seq_map_break:n
```

`\__bnvs_foreach_P:nNTF` `\__bnvs_foreach_P:nNTF {⟨id⟩} ⟨function:nn⟩ {⟨yes:⟩} {⟨no:⟩}`
`\__bnvs_foreach_P:nnTF` `\__bnvs_foreach_P:nnTF {⟨id⟩} {⟨:nn code⟩} {⟨yes:⟩} {⟨no:⟩}`

If ⟨*id*⟩ is a declared identifier, execute ⟨*function:nn*⟩ or ⟨*code*⟩ for each combination of ⟨*id*⟩ and its associate ⟨*path*⟩s. Both are the arguments passed to ⟨*function:nn*⟩ or ⟨*:nn code*⟩ (through ##1 and ##2).

```
\BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
  \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
    \seq_map_inline:cn {
      \__bnvs_g_I:w { #1 }
    } {
      #2 { #1 } { ##1 }
    }
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
\BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
  \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
    \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
    \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
      \BNVS_foreach_P_nn:nn { #1 } { ##1 }
    }
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

`\__bnvs_foreach_K:nnN` `\__bnvs_foreach_K:nnN {⟨id⟩} {⟨path⟩} ⟨function:nnn⟩`
`\__bnvs_foreach_K:nnn` `\__bnvs_foreach_K:nnn {⟨id⟩} {⟨path⟩} {⟨:nnn code⟩}`

Execute the ⟨*function:nnn*⟩ or the ⟨*:nnn code*⟩ for each of ⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩ combination. These are the arguments to the function or code argument.

```
\BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
  \seq_if_exist:NT #1 {
    \seq_map_inline:Nn #1 {
      #4 { #2 } { #3 } { ##1 }
    }
  }
}
\BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
  \exp_args:Nc \__bnvs_foreach_K:NnnN
    { \__bnvs_g_IP:w { #1 } { #2 } }
    { #1 } { #2 } #3
}
\BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
  \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
  \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
}
```

__bnvs_foreach_IP:N __bnvs_foreach_IP:N ⟨*function:nn*⟩
__bnvs_foreach_IP:n __bnvs_foreach_IP:n {⟨*:nn code*⟩}

Execute the ⟨*function:nn*⟩ or the ⟨*:nn code*⟩ for each ⟨*id*⟩!⟨*path*⟩ combination.

```
\BNVS_new:cpn { foreach_IP:N } #1 {
  \__bnvs_foreach_I:n {
    \__bnvs_foreach_P:nNT { ##1 } #1 { }
  }
}

\BNVS_new:cpn { foreach_IP:NT } #1 #2 {
  \__bnvs_foreach_I:n {
    \__bnvs_foreach_P:nNT { ##1 } #1 { #2 }
  }
}

\BNVS_new:cpn { foreach_IP:n } #1 {
  \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
  \__bnvs_foreach_I:n {
    \__bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
  }
}
```

6.7.5 Low level IP model functions

6.7.6 Low level I model functions

6.7.7 Getting

__bnvs_if_get:nnnc*TF* __bnvs_if_get:nnncTF {⟨*id*⟩} {⟨*path*⟩} {⟨*key*⟩} {⟨*ans*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_if_spec:nnnc*TF* __bnvs_if_spec:nnncTF {⟨*id*⟩} {⟨*path*⟩} {⟨*key*⟩} {⟨*ans*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_if_get:nnc*TF* __bnvs_if_get:nncTF {⟨*id*⟩} {⟨*path*⟩} {⟨*ans*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_if_get:vvc*TF*

The __bnvs_if_get:nnnc... variant puts what was stored for ⟨*id*⟩!⟨*path*⟩/⟨*key*⟩ into the ⟨*ans*⟩ variable, if any, then executes ⟨*yes:*⟩. Otherwise executes ⟨*no:*⟩ without changing the contents of the ⟨*ans*⟩ tl variable.

__bnvs_if_get:nncTF is a convenient shortcut for __bnvs_if_get:nnncTF when ⟨*key*⟩ and ⟨*ans*⟩ happen to be the same.

The _spec: variant is similar except that it uses ⟨*id*⟩!⟨*path*⟩/⟨*key*⟩ combinations when available or !⟨*path*⟩/⟨*key*⟩ otherwise.

```
\BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
    \BNVS_tl_use:Nc \cs_set_eq:Nc
      { #4 } { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
    \prg_return_true:
  } {
    \prg_return_false:
  }
}

\BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
```

```
    \tl_set_eq:cc { \BNVS_l:cn { #4 } { tl } } {
      \__bnvs_g_IPK:w { #1 } { #2 } { #3 }
    }
    \prg_return_true:
  } {
    \tl_if_empty:nTF { #1 } {
      \prg_return_false:
    } {
      \__bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
        \prg_return_true: \prg_return_false:
    }
  }
}

\BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:Nvv \__bnvs_if_get:nncTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
```

`\__bnvs_if_resolved:nnnc`*TF* `\__bnvs_if_resolved:nnncTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨ans⟩} {⟨yes:⟩} {⟨no:⟩}`

Execute ⟨*yes:*⟩ if resolution succeeded for either ⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩ or the fallback `!`⟨*path*⟩`/`⟨*key*⟩, otherwise execute ⟨*no:*⟩.

```
\BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
    \__bnvs_is_will_gset:cTF { #3 }
    \prg_return_true: \prg_return_false:
  } {
    \tl_if_empty:nTF { #1 } {
      \prg_return_false:
    } {
      \__bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
        \prg_return_true: \prg_return_false:
    }
  }
}
```

`\__bnvs_require:nnnc` `\__bnvs_require:nnnc {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨ans⟩}`
`\__bnvs_require:nnc` `\__bnvs_require:nnc {⟨id⟩} {⟨path⟩} {⟨ans⟩}`
`\__bnvs_require:vvc`

Calls `\__bnvs_if_get:nnncTF`, does nothing on success and, on failure, does nothing or raise when in debug mode. `\__bnvs_require:nnc` is a shortcut for the more qualified function `\__bnvs_require:nnnc` when ⟨*key*⟩ and ⟨*ans*⟩ happen to be identical.

```
\BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
  \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
    \BNVS_fatal_unreachable:
  }
}
```

__bnvs_gunset:nn
__bnvs_gunset:(nv|vn|vv)

__bnvs_gunset:nn {⟨*id*⟩} {⟨*path*⟩}

Removes the specifications for the ⟨*id*⟩!⟨*path*⟩ and all possible keys combination.

```
\BNVS_new:cpn { gunset:NNNnn } #1 #2 #3 #4 #5 {
  \seq_map_inline:Nn #1 {
    \__bnvs_gunset:nnn { #4 } { #5 } { ##1 }
  }
  \cs_undefine:N #1
  \__bnvs_seq_gremove_once:Nn #2 { #5 }
  \seq_if_empty:NT #2 {
    \cs_undefine:N #2
    \__bnvs_seq_gremove_once:Nn #3 { #4 }
    \cs_undefine:c { __bnvs_gset@ #4 :nnn }
  }
}
\BNVS_new:cpn { gunset:Nnn } #1 #2 #3 {
  \seq_map_inline:Nn #1 {
    \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
      \exp_args:Nc \__bnvs_gunset:NNNnn
        { \__bnvs_g_IP:w  { #2 } { ##1 } }
        #1 \g__bnvs_I_seq { #2 } { ##1 }
      \cs_undefine:c { __bnvs_gset@{ #2 } { ##1 }:nn }
    }
  }
}
\BNVS_new:cpn { gunset:nn } #1 #2 {
  \exp_args:Nc
  \__bnvs_gunset:Nnn { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
}

\BNVS_new:cpn { gunset:nv } #1 {
  \BNVS_tl_use:nv { \__bnvs_gunset:nn { #1 } }
}

\BNVS_new:cpn { gunset:vn } {
  \BNVS_tl_use:cv { gunset:nn }
}

\BNVS_new:cpn { gunset:vv } {
  \BNVS_tl_use:Nvv \__bnvs_gunset:nn
}
```

__bnvs_gunset_deep:nn
__bnvs_gunset_deep:vv

__bnvs_gunset_deep:nn {⟨*id*⟩} {⟨*path*⟩}

Removes the specifications for each existing ⟨*id*⟩!⟨*path*⟩.⟨*subpath*⟩ combination.

```
\BNVS_new:cpn { gunset_deep:Nnn } #1 #2 #3 {
  \seq_if_exist:NT #1 {
    \seq_map_inline:Nn #1 {
```

`#3` is exactly `##1` or start with `##1.`, which is equivalent to `#3.` starts with `##1.`:

```
      \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
        \__bnvs_gunset:nn { #2 } { ##1 }
      }
    }
  }
}

\BNVS_new:cpn { gunset_deep:nn } #1 #2 {
  \exp_args:Nc \__bnvs_gunset_deep:Nnn
    { \__bnvs_g_I:w { #1 } }
    { #1 } { #2 }
}

\BNVS_new:cpn { gunset_deep:vv } {
  \BNVS_tl_use:Nvv \__bnvs_gunset_deep:nn
}
```

`\__bnvs_gunset:n` `\__bnvs_gunset:n {⟨id⟩}`
`\__bnvs_gunset:` `\__bnvs_gunset:`

Removes the specifications for the ⟨*id*⟩ and all possible tag combination. The second does the same for all possible ⟨*id*⟩.

```
\BNVS_new:cpn { gunset:NNNn } #1 #2 #3 #4 {
  \cs_if_exist:NT #1 {
    \cs_undefine:N #1
    \seq_map_inline:Nn #2 {
      \__bnvs_gunset:nn { #4 } { ##1 }
    }
    \cs_undefine:N #2
    \__bnvs_seq_gremove_once:Nn #3 { #4 }
  }
}
\BNVS_new:cpn { gunset:n } #1 {
  \exp_args:Ncc \__bnvs_gunset:NNNn
    { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
    \g__bnvs_I_seq { #1 }
}
\BNVS_new:cpn { gunset: } {
  \seq_map_inline:Nn \g__bnvs_I_seq {
    \__bnvs_gunset:n { ##1 }
  }
}
```

`\__bnvs_VS_gunset:` `\__bnvs_VS_gunset:`

Removes any value for the "V" and "S" keys and any ⟨*id*⟩`!`⟨*path*⟩ combination.

```
\BNVS_new:cpn { VS_gunset: } {
  \__bnvs_foreach_IP:n {
    \__bnvs_gunset:nnn { ##1 } { ##2 } V
    \__bnvs_gunset:nnn { ##1 } { ##2 } S
  }
}
```

6.7.8 Circular dependencies

__bnvs_will_gset:nnn `\__bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}`

To manage circular dependencies. After this commmand, `\__bnvs_is_will_gset:nnnTF` is true for the same arguments.

```
\BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
  \__bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
}
```

__bnvs_is_will_gset:c*TF* `\__bnvs_is_will_gset:cTF {⟨in⟩} {⟨yes:⟩} {⟨no:⟩}`

To manage circular dependencies. See `\__bnvs_will_gset:nnn` above.

```
\__bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}
...
\__bnvs_if_get:nnncT {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨in⟩} {
  ...
  \__bnvs_is_will_gset:cTF
    {⟨in⟩}
    {⟨yes:⟩}
    {⟨no:⟩}
  ...
}
```

```
\BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
  \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

6.7.9 Spring cleaning

__bnvs_gclear:nn `\__bnvs_gclear:nn {⟨id⟩} {⟨path⟩}`
__bnvs_gclear:nv `\__bnvs_gclear:n {⟨id⟩}`
__bnvs_gclear: `\__bnvs_gclear:`
__bnvs_greset:nn `\__bnvs_greset:nn {⟨id⟩} {⟨path⟩}`
__bnvs_greset:nv `\__bnvs_greset:n {⟨id⟩}`
__bnvs_greset: `\__bnvs_greset:`

The first ones clear out every data stored for ⟨*id*⟩!⟨*path*⟩, including deeper and registration data. When ⟨*path*⟩ is omitted, any known path is considered. When ⟨*id*⟩ is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for key =) is left untouched.

```
\BNVS_new:cpn { greset:nn } #1 #2 {
```

```
  \__bnvs_foreach_P:nnT { #1 } {
    \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
      \__bnvs_foreach_K:nnn { #1 } { ##2 } {
        \tl_if_in:nnF { \q_bnvs ####3 } { \q_bnvs = } {
          \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
        }
      }
    }
  } { }
}
```

Clears out every data associated to ⟨*id*⟩!⟨*path*⟩, and any ⟨*id*⟩!⟨*path*⟩.⟨*subpath*⟩.

```
\BNVS_new:cpn { gclear:nn } #1 #2 {
  \__bnvs_foreach_P:nnT { #1 } {
    \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
      \__bnvs_foreach_K:nnn { #1 } { ##2 } {
        \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
          \__bnvs_gset:nnnn  { #1 } { ##2 } { ####3 } { 0 }
        } {
          \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
        }
      }
    }
  } { }
}
\BNVS_new:cpn { gclear:n } #1 {
  \__bnvs_foreach_P:nNT { #1 } \__bnvs_gclear:nn {}
}
\BNVS_new:cpn { gclear: } {
  \__bnvs_foreach_IP:N \__bnvs_gclear:nn
}
\BNVS_new:cpn { greset:n } #1 {
  \__bnvs_foreach_P:nNT { #1 } \__bnvs_greset:nn {}
}
\BNVS_new:cpn { greset: } {
  \__bnvs_foreach_IP:N \__bnvs_greset:nn
}
```

__bnvs_gclear_resolved:nn
__bnvs_gclear_resolved:nv

__bnvs_gclear_resolved:nn {⟨*id*⟩} {⟨*path*⟩}

Clear the resolved data, aka any key that does not start with "=".

```
\BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
  \__bnvs_foreach_IP:nnn { #1 } { #2 } {
    \tl_if_in:nnF { @ ##1 } { @ = } {
      \__bnvs_gunset:nnn { #1 } { #2 } { ##1 }
    }
  }
}
\BNVS_new:cpn { gclear_resolved:nv } #1 {
  \BNVS_tl_use:nv {
    \__bnvs_gclear_resolved:nn { #1 }
  }
}
```

6.7.10 The provide mode

\l__bnvs_provide_bool Manage the provide mode.

```
\bool_new:N \l__bnvs_provide_bool
```

(*End of definition for* `\l__bnvs_provide_bool`.)

```
\BNVS_new:cpn { provide_ON: } {
  \__bnvs_set_true:c { provide }
}
\BNVS_new:cpn { provide_OFF: } {
  \__bnvs_set_false:c { provide }
}
\__bnvs_provide_OFF:
```

__bnvs_if_should_provide:nnn*TF* __bnvs_if_should_provide:nnnTF {⟨*id*⟩} {⟨*path*⟩} {⟨*key*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_if_should_provide:nn*TF* __bnvs_if_should_provide:nnTF {⟨*id*⟩} {⟨*path*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

Execute ⟨*yes:*⟩ when in provide mode and ⟨*id*⟩!⟨*path*⟩/⟨*key*⟩ is known, ⟨*no:*⟩ otherwise. When not specified, ⟨*key*⟩ is "=".

```
\BNVS_new_conditional:cpnn { if_should_provide:nnn } #1 #2 #3 { T, F, TF } {
  \__bnvs_if:cTF { provide } {
      \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
      \prg_return_false:
    } {
      \prg_return_true:
    }
  } {
    \prg_return_true:
  }
}
\BNVS_new_conditional:cpnn { if_should_provide:nn } #1 #2 { T, F, TF } {
  \__bnvs_if_should_provide:nnnTF { #1 } { #2 } =
    \prg_return_true: \prg_return_false:
}
```

__bnvs_is_gset_and_provide:nnn*TF* __bnvs_is_gset_and_provide:nnnTF {⟨*id*⟩} {⟨*path*⟩} {⟨*key*⟩}
__bnvs_is_gset_and_provide:(nvn|vvn)*TF* {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_is_gset_and_provide:nn*TF* __bnvs_is_gset_and_provide:nnTF {⟨*id*⟩} {⟨*path*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

Execute ⟨*yes:*⟩ when in provide mode and ⟨*id*⟩!⟨*path*⟩/⟨*key*⟩ is known, ⟨*no:*⟩ otherwise. When not specified, ⟨*key*⟩ is "=".

```
\BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
        #1 #2 #3 { T, F, TF } {
  \__bnvs_if:cTF { provide } {
    \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  } {
```

```
    \prg_return_false:
  }
}
\BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
                           #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:nv {
    \__bnvs_is_gset_and_provide:nnnTF { #1 }
  } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { is_gset_and_provide:vvn }
        #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
```

__bnvs_gprovide:TnnnnF
__bnvs_gprovide:TvnnnF
__bnvs_gprovide:TvvnnF

__bnvs_gprovide:TnnnnF {⟨*pre code*⟩} {⟨*id*⟩} {⟨*path*⟩} {⟨*key*⟩} {⟨*value*⟩} {⟨*no:*⟩}

Execute ⟨*no:*⟩ exclusively when not in provide mode. Does nothing when something was set for the ⟨*id*⟩!⟨*path*⟩/⟨*key*⟩ combination. Execute ⟨*pre code*⟩ before setting.

```
\BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
  \__bnvs_if:cTF { provide } {
    \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
      #1
      \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
    }
  }
}
\BNVS_new:cpn { gprovide:TvnnnF } #1 {
  \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
}
\BNVS_new:cpn { gprovide:TvvnnF } #1 {
  \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
}
```

6.7.11 Initialize

Here, ⟨*simple definition*⟩ contains no ⟨*key*⟩-⟨*value*⟩ material, even implicit.

__bnvs_ginit:nnn
__bnvs_ginit:(nnv|vvn)

__bnvs_ginit:nnn {⟨*id*⟩} {⟨*path*⟩} {⟨*simple definition*⟩}

This function supports provide mode. Somehow a shortcut to __bnvs_gset:nnnn with key "=". If ⟨*path*⟩ has more than one component, each intermediate path is modified or defined appropriately.

```
\BNVS_new:cpn { ginit:nnn } #1 #2 #3 {
  \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
    \__bnvs_greset:nn { #1 } { #2 }
    \__bnvs_gset:nnnn { #1 } { #2 } = { #3 }
  }
}
\BNVS_new:cpn { ginit:nnv } #1 #2 {
  \BNVS_tl_use:nv { \__bnvs_ginit:nnn { #1 } { #2 } }
}
```

```
\BNVS_new:cpn { ginit:vvn } {
  \BNVS_tl_use:Nvv \__bnvs_ginit:nnn
}
```

`\__bnvs_ginit_ITND:w` `\__bnvs_ginit_ITND:w {⟨id⟩} {⟨path⟩} {⟨index⟩} {⟨simple definition⟩}`

These functions support provide mode. Somehow a shortcut to `\__bnvs_gset:nnnn` with key ⟨*index*⟩. When ⟨*index*⟩ is provided, it is already normalized. If ⟨*id*⟩`!`⟨*path*⟩`/=` is not set, it is initialized from ⟨*definition*⟩ shifted appropriately relative to ⟨*index*⟩.

```
\BNVS_new:cpn { ginit_ITND:w } #1 #2 #3 #4 {
  \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } { #3 } {
    \__bnvs_gunset:nnn { #1 } { #2 } {  #3 }
    \__bnvs_gset:nnnn  { #1 } { #2 } { =#3 } { #4 }
  }
  \__bnvs_is_gset:nnnF { #1 } { #2 } = {
    \__bnvs_gset:nnne { #1 } { #2 } = {
      \exp_not:n { #4 } - \int_eval:n { #3 - 1 }
    }
  }
}
```

`\__bnvs_gdef_IPKD:w` `\__bnvs_gdef_IPKD:w {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨simple definition⟩}`

This function calls `\__bnvs_gdef_by_key:nnn` with ⟨*path*⟩`.`⟨*key*⟩ as second argument.

```
\BNVS_new:cpn { gdef_IPKD:w } #1 #2 #3 #4 {
  \__bnvs_gdef_by_key:nnn { #1 } { #2.#3 } { #4 }
}
```

6.7.12 Provide mode

`\__bnvs_is_provided:nnn`*TF* `\__bnvs_is_provided:nnnTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨yes:⟩} {⟨no:⟩}`

If ⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩ has been successfully resolved and not unset since then, execute ⟨*yes:*⟩. Otherwise execute ⟨*no:*⟩.

```
\tl_new:N \l__bnvs_is_provided_tl
\BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 }  { is_provided } {
    \__bnvs_is_will_gset:cTF { is_provided } {
      \prg_return_false:
    } {
      \prg_return_true:
    }
  } {
    \prg_return_false:
  }
}
```

6.8 Regular expressions

\c__bnvs_S_regex This regular expression is used for both short names and dot path components. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit. It must contain one uppercase letter.

```
\regex_const:Nn \c__bnvs_S_regex {
  (?:[a-z_][a-z0-9_]*)?[A-Z][[:alnum:]_]*
}
```

(*End of definition for* \c__bnvs_S_regex.)

\c__bnvs_R_regex A possibly empty list of .⟨*short name*⟩ items representing a path.

```
\regex_const:Nn \c__bnvs_R_regex {
  (?: \. \ur{c__bnvs_S_regex} )*
}
```

(*End of definition for* \c__bnvs_R_regex.)

\c__bnvs_U_regex An ⟨*attribute*⟩, no uppercase variable.

```
\regex_const:Nn \c__bnvs_U_regex {
  [a-z_][a-z0-9_]*
}
```

(*End of definition for* \c__bnvs_U_regex.)

\c__bnvs_O_regex A possibly empty list of .⟨*attributes*⟩ items representing a path.

```
\regex_const:Nn \c__bnvs_O_regex {
  (?: \. \ur{c__bnvs_U_regex} )*
}
```

(*End of definition for* \c__bnvs_O_regex.)

\c__bnvs_A_N_Z_regex Signed integer with two capture groups.

(*End of definition for* \c__bnvs_A_N_Z_regex.)

```
\regex_const:Nn \c__bnvs_A_N_Z_regex {
  \A \. \s* [-+\s]* \d+ \Z
}
```

\c__bnvs_A_index_Z_regex Signed integer with no capture groups.

(*End of definition for* \c__bnvs_A_index_Z_regex.)

```
\regex_const:Nn \c__bnvs_A_index_Z_regex { \A [-+\s]* \d + \Z }
```

\c__bnvs_A_reserved_Z_regex Reserved words.

(*End of definition for* \c__bnvs_A_reserved_Z_regex.)

```
\regex_const:Nn \c__bnvs_A_reserved_Z_regex {
  \A_*[a-z][_a-z0-9]*\Z
}
```

\c__bnvs_A_IKPN_Z_regex Matches the whole string, split into ⟨*id*⟩ and ⟨*path*⟩.

(*End of definition for* \c__bnvs_A_IKPN_Z_regex.)

1: The full match,

```
\regex_const:Nn \c__bnvs_A_IKPN_Z_regex {
```

2,3: the optional frame ⟨*id*⟩ and `!`

```
    \A (?: ( \ur{c__bnvs_S_regex} )? (!) )?
```

4: The tag (short name and dotted path)

```
    ( \ur{c__bnvs_S_regex} \ur{c__bnvs_R_regex} ? )
```

5: a signed number that will be normalized.

```
    (?: \. \s* ([-+]? \s* \d+ ) )? \Z
  }
```

\c__bnvs_A_IKSRN_Z_regex Matches the whole string, split into ⟨*id*⟩ and ⟨*tag*⟩.
(*End of definition for* `\c__bnvs_A_IKSRN_Z_regex`.)

1: The full match,

```
\regex_const:Nn \c__bnvs_A_IKSRN_Z_regex {
```

2,3: the optional frame ⟨*id*⟩ and `!`

```
  \A (?: ( \ur{c__bnvs_S_regex} )? (!) )?
```

4: The short name

```
  ( \ur{c__bnvs_S_regex} )
```

5: The dotted path

```
  ( \ur{c__bnvs_R_regex} )?
```

6: a signed number that will be normalized.

```
  (?: \s* \. \s* ([-+]? \s* \d+ ) )? \Z
}
```

\c__bnvs_A_SR_Z_regex Matches the whole string.
(*End of definition for* `\c__bnvs_A_SR_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_SR_Z_regex {
```

1: The full match,

2: the frame ⟨*id*⟩

```
  \A ( \ur{c__bnvs_S_regex} | [-+]? \d+ )
```

3: The dotted path.

```
  ( (?: \. \ur{c__bnvs_S_regex} | \. [-+]? \d+ )* ) \Z
}
```

\c__bnvs_A_P_Z_regex Matches the whole string.
(*End of definition for* `\c__bnvs_A_P_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_P_Z_regex {
  \A \ur{c__bnvs_S_regex} \ur{c__bnvs_R_regex} \Z
}
```

\c__bnvs_colons_regex For ranges defined by a colon syntax. One catching group for more than one colon.

```
\regex_const:Nn \c__bnvs_colons_regex { :(:+)? }
```

(*End of definition for* `\c__bnvs_colons_regex`.)

\c__bnvs_A_VAZLLZZL_Z_regex Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.
(*End of definition for* `\c__bnvs_A_VAZLLZZL_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
  \A \s* (?:
```

- 2 → V

```
    ( [^:]+? )
```

- 3, 4, 5 → A : Z? or A :: L?

```
    | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
```

- 6, 7 → ::(L:Z)?

```
    | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
```

- 8, 9 → :(Z::L)?

```
    | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
  )
  \s* \Z
}
```

6.9 End group setters

```
\cs_new:Npn \BNVS_after_end_tl_put_right:cv #1 {
  \BNVS_after_end:n {
    \__bnvs_tl_put_right:cn { #1 }
  }
  \BNVS_tl_use:Nv \BNVS_after_end_braced:n
}
\cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
  \BNVS_after_end_tl_put_right:cv { #1 } { #2 }
  \BNVS_end:
}
\cs_new:Npn \BNVS_after_end_tl_gset:nnnv #1 #2 #3 {
  \BNVS_after_end:n {
    \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
  }
  \BNVS_tl_use:Nv \BNVS_after_end_braced:n
}
\cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
  \BNVS_after_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
  \BNVS_end:
}
\cs_new:Npn \BNVS_after_end_tl_set:cv #1 {
  \BNVS_after_end:n {
    \__bnvs_tl_set:cn { #1 }
  }
  \BNVS_tl_use:Nv \BNVS_after_end_braced:n
}
\cs_new:Npn \BNVS_end_tl_set:cv #1 #2 {
  \BNVS_after_end_tl_set:cv { #1 } { #2 }
  \BNVS_end:
}
\cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
  \BNVS_tl_use:nv {
    #1
    \__bnvs_tl_set:cn { #2 }
  }
}
\cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
  \BNVS_tl_set:ncv {
    \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
  } { #4 }
}
```

```
\cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
  \BNVS_tl_set:ncv {
    \BNVS_tl_set:ncv {
      \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
    } { #4 } { #5 }
  } { #6 }
}
\cs_new:Npn \BNVS_after_end_int_set:cv #1 {
  \BNVS_after_end:n {
    \__bnvs_int_set:cn { #1 }
  }
  \BNVS_tl_use:Nv \BNVS_after_end_braced:n
}
\cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
  \BNVS_after_end_int_set:cv { #1 } { #2 }
  \BNVS_end:
}
```

6.10 beamer.cls interface

Work in progress.

```
\RequirePackage{keyval}
\define@key{beamerframe}{beanoves~id}[]{
  \tl_set:Nx \l__bnvs_J_tl { #1 }
}
\bool_new:N \l__bnvs_in_frame_bool
\bool_set_false:N \l__bnvs_in_frame_bool
\AddToHook{env/beamer@frameslide/before}{
  \__bnvs_greset_call:
  \__bnvs_VS_gunset:
  \__bnvs_set_true:c { in_frame }
}
\AddToHook{env/beamer@frameslide/after}{
  \__bnvs_set_false:c { in_frame }
}
```

6.11 Utilities

6.11.1 Split utilities

__bnvs_if_regex_split:cncTF __bnvs_if_regex_split:cncTF {⟨regex core⟩} {⟨expression⟩} ⟨seq core⟩
__bnvs_if_regex_split:cnTF {⟨yes:⟩} {⟨no:⟩}
__bnvs_if_regex_split:cnTF {⟨regex core⟩} {⟨expression⟩} {⟨yes:⟩} {⟨no:⟩}

These are shortcuts to \regex_split:NnNTF with the split sequence as last N argument.

```
\BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
  } { #3 } \prg_return_true: \prg_return_false:
}
```

```
\BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
  } { split } \prg_return_true: \prg_return_false:
}
```

__bnvs_split_pop:	__bnvs_split_pop:
__bnvs_split_pop:c	__bnvs_split_pop:c {⟨var$_1$⟩}
__bnvs_split_pop:cc	__bnvs_split_pop:cc {⟨var$_1$⟩} {⟨var$_2$⟩}
__bnvs_split_pop:ccc	__bnvs_split_pop:ccc {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩}
__bnvs_split_pop:cccc	__bnvs_split_pop:cccc {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩} {⟨var$_4$⟩}
__bnvs_split_pop:ccccc	__bnvs_split_pop:ccccc {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩} {⟨var$_4$⟩} {⟨var$_5$⟩}

```
\tl_new:N \l__bnvs_split_pop_tl
\BNVS_new:cpn { split_pop: } {
  \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
}
\BNVS_new:cpn { split_pop:c } #1 {
  \BNVS_tl_use:nc {
    \seq_pop_left:NN \l__bnvs_split_seq
  } { #1 }
}
\BNVS_new:cpn { split_pop:cc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:c
}
\BNVS_new:cpn { split_pop:ccc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:cc
}
\BNVS_new:cpn { split_pop:cccc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:ccc
}
\BNVS_new:cpn { split_pop:ccccc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:cccc
}
\BNVS_new_conditional:cpnn { split_if_pop_left:c } #1 { T, F, TF } {
  \__bnvs_seq_pop_left:ccTF { split } { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

6.11.2 Match utilities

__bnvs_match_if_once:NnTF __bnvs_match_if_once:NnTF ⟨*regex variable*⟩ {⟨*expression*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_match_if_once:NvTF __bnvs_match_if_once:nnTF {⟨*regex*⟩} {⟨*expression*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_match_if_once:nnTF These are shortcuts to

- `\regex_match_if_once:NnNTF` with the `match` sequence as N argument
- `\regex_match_if_once:nnNTF` with the `match` sequence as N argument
- `\regex_split:NnNTF` with the `split` sequence as last N argument

```
\BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
  \BNVS_use:ncn {
    \regex_extract_once:NnNTF #1 { #3 }
  } { #2 } { seq } \prg_return_true: \prg_return_false:
}
```

\l__bnvs_match_seq Local storage for the match result.
(*End of definition for* `\l__bnvs_match_seq`.)

```
\BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
  \BNVS_use:ncn {
    \regex_extract_once:NnNTF #1 { #2 }
  } { match } { seq } \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_tl_use:nv {
      \regex_extract_once:NnNTF #1
    } { #3 }
  } { #2 } \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_tl_use:nv {
      \regex_extract_once:NnNTF #1
    } { #2 }
  } { match } \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
  \BNVS_seq_use:nc {
    \regex_extract_once:nnNTF { #1 } { #2 }
  } { match } \prg_return_true: \prg_return_false:
}
\BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
  \BNVS_tl_use:nc {
    \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
  } { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

__bnvs_match_pop: __bnvs_match_pop:
__bnvs_match_pop:c __bnvs_match_pop:c {⟨var$_1$⟩}
__bnvs_match_pop:cc __bnvs_match_pop:cc {⟨var$_1$⟩} {⟨var$_2$⟩}
__bnvs_match_pop:ccc __bnvs_match_pop:ccc {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩}
__bnvs_match_pop:cccc __bnvs_match_pop:cccc {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩} {⟨var$_4$⟩}
__bnvs_match_pop:ccccc __bnvs_match_pop:ccccc {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩} {⟨var$_4$⟩} {⟨var$_5$⟩}

```
\tl_new:N \l__bnvs_match_pop_tl
\BNVS_new:cpn { match_pop: } {
  \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
}
\BNVS_new:cpn { match_pop:c } #1 {
  \BNVS_tl_use:nc {
    \seq_pop_left:NN \l__bnvs_match_seq
  } { #1 }
}
\BNVS_new:cpn { match_pop:cc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:c
}
\BNVS_new:cpn { match_pop:ccc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:cc
}
\BNVS_new:cpn { match_pop:cccc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:ccc
}
\BNVS_new:cpn { match_pop:ccccc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:cccc
}
```

\c__bnvs_A_IKSR_Z_regex A qualified dotted name is the qualified name of an overlay set possibly followed by a dotted path. Matches the whole string. Four capture groups.
(*End of definition for* \c__bnvs_A_IKSR_Z_regex.)

```
\regex_const:Nn \c__bnvs_A_IKSR_Z_regex {
```

1: the frame ⟨*id*⟩

2: the exclamation mark

```
  \A (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: The short name.

```
  ( \ur{c__bnvs_S_regex} )
```

4: the remaining part of the path, if any.

```
  ( \ur{c__bnvs_R_regex} ) \Z
}
```

__bnvs_if_ISR:n*TF* __bnvs_if_ISR:nTF {⟨*name*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_if_ISR:nnTF {⟨*root*⟩} {⟨*relative*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

If ⟨*name*⟩ is a reference, put the frame id it defines, or the current frame id, into `I`, the short name into `S`, the dotted path into `R` then execute ⟨*yes:*⟩. Otherwise execute ⟨*no:*⟩.

The `J` variable is updated.

```
\BNVS_new_conditional:cpnn { if_ISR:n } #1 { T, F, TF } {
  \BNVS_begin:
  \__bnvs_match_if_once:NnTF \c__bnvs_A_IKSR_Z_regex { #1 } {
    \__bnvs_match_pop:
    \__bnvs_match_pop:cccc IKSR
    \cs_set:Npn \BNVS_if_ISR:nnn ##1 ##2 ##3 {
      \BNVS_end:
      \__bnvs_tl_set:cn I { ##1 }
      \__bnvs_tl_set:cn S { ##2 }
      \__bnvs_tl_set:cn R { ##3 }
    }
    \__bnvs_tl_if_empty:cTF K {
      \BNVS_tl_use:Nvvv \BNVS_if_ISR:nnn J
    } {
      \BNVS_tl_use:Nvvv \BNVS_if_ISR:nnn I
    } S R
    \__bnvs_tl_set:cv T R
    \__bnvs_tl_put_left:cv T S
    \__bnvs_tl_set:cv J I
    \prg_return_true:
  } {
    \BNVS_end:
    \prg_return_false:
  }
}
```

`\__bnvs_if_ITN:nTF` `\__bnvs_if_ITN:nTF {⟨name⟩} {⟨yes:⟩} {⟨no:⟩}`

Called from `\__bnvs_root_parsed:n` and `\__bnvs_root_parsed:nn`. If ⟨*name*⟩ is a reference, put the frame id it defines, or the current frame id (the value of `J`), into `I`, the tag into `T` and the normalized number index into `N`, if any, then execute ⟨*yes:*⟩. Otherwise execute ⟨*no:*⟩.

The `J` variable is updated.

```
\BNVS_new_conditional:cpnn { if_ITN:n } #1 { T, F, TF } {
  \BNVS_begin:
  \__bnvs_match_if_once:NnTF \c__bnvs_A_IKPN_Z_regex { #1 } {
    \__bnvs_match_pop:
    \__bnvs_match_pop:cccc IKTN
    \cs_set:Npn \BNVS_if_ITN:w ##1 ##2 ##3 {
      \BNVS_end:
      \__bnvs_tl_set:cn I { ##1 }
      \__bnvs_tl_set:cn T { ##2 }
      \__bnvs_tl_set:cn N { ##3 }
      \__bnvs_normalize_N:
    }
    \__bnvs_tl_if_empty:cTF K {
      \BNVS_tl_use:Nvvv \BNVS_if_ITN:w J
    } {
      \BNVS_tl_use:Nvvv \BNVS_if_ITN:w I
    } T N
    \__bnvs_tl_set:cv J I
```

```
    \prg_return_true:
  } {
    \BNVS_end:
    \prg_return_false:
  }
}
\BNVS_undefine:c { if_ITN:n }
```

6.11.3 Path utilities

\c__bnvs_A_PN_Z_regex

Matches the whole string.
(*End of definition for* `\c__bnvs_A_PN_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_PN_Z_regex {
```

1: The full match,

2: the path

```
  \A ( \ur{c__bnvs_S_regex} \ur{c__bnvs_R_regex} )
```

3: The sign and the value of the trailing digits component.

```
  (?: \. \s * ( [-+]? \s* \d+ ) )? \Z
}
```

__bnvs_if_PN:nTTF `\__bnvs_if_PN:nTTF {⟨path⟩} {⟨yes:n⟩} {⟨yes:nn⟩} {⟨no:⟩}`

If ⟨*path*⟩ is a correct path with no trailing number, execute ⟨*yes:n*⟩ with the whole ⟨*path*⟩ as argument. If ⟨*path*⟩ is a correct path with a trailing number, executes ⟨*yes:nn*⟩ with the whole ⟨*path*⟩ but its last component as first argument and the normalized number as second arguments. If ⟨*path*⟩ is not a correct path, execute ⟨*no:*⟩.

```
\BNVS_new:cpn { if_PN:nTTF } #1 #2 #3 #4 {
  \BNVS_begin:
  \__bnvs_match_if_once:NnTF \c__bnvs_A_PN_Z_regex { #1 } {
    \__bnvs_match_pop:
    \__bnvs_match_pop:cc PN
    \__bnvs_tl_if_empty:cTF { N } {
      \cs_set:Npn \BNVS_if_path_nTTF:w ##1 { \BNVS_end: #2 }
      \BNVS_tl_use:Nv \BNVS_if_path_nTTF:w P
    } {
      \__bnvs_normalize_N:
      \cs_set:Npn \BNVS_if_path_nTTF:w ##1 ##2 { \BNVS_end: #3 }
      \BNVS_tl_use:Nvv \BNVS_if_path_nTTF:w P N
    }
  } {
    \BNVS_end:
    #4
  }
}
```

6.11.4 Conversions

`\__bnvs_convert:nc` `\__bnvs_convert:nc {⟨definitions⟩} {⟨var⟩}`

Expand and convert ⟨*definitions*⟩ into ⟨*var*⟩, on return the variable only contains characters with category code other.

```
\BNVS_new:cpn { convert:nc } #1 #2 {
  \__bnvs_tl_set:ce { #2 } { \exp_args:Nc \cs_to_str:N { #1 } }
}
```

`\__bnvs_prepare:nnc` `\__bnvs_prepare:nnc {⟨key⟩} {⟨error⟩} {⟨var⟩}`

Used by `\__bnvs_prepare_crl:nc`, `\__bnvs_prepare_sqr:nc` and `\__bnvs_prepare_rnd:nc`. Implementation detail: local functions `\BNVS:N` and `\BNVS_loop:`.

```
\BNVS_new:cpn { prepare:nnc } #1 #2 #3 {
  \cs_set:Npn \BNVS:N ##1 {
    \cs_set:Npn \BNVS_loop: {
      \exp_args:Nc \regex_replace_all:NnNT { c__bnvs_#1_n_regex } {
        \c { BNVS_#1:n } \cB \{ \1 \cE \}
      } ##1 {
        \BNVS_loop:
      }
    }
    \BNVS_loop:
    \exp_args:Nc \regex_match:NVT { c__bnvs_#1_regex } ##1 {
      \BNVS_error:n { #2 }
    }
  }
  \BNVS_tl_use:Nc \BNVS:N { #3 }
}
```

`\c__bnvs_crl_n_regex` Used by `\__bnvs_prepare_crl:c` and `\__bnvs_resolve_prepare:nc`.

```
\regex_const:Nn \c__bnvs_crl_n_regex { \cO \{ ( .*? ) \cO \} }
```

(*End of definition for* `\c__bnvs_crl_n_regex`.)

`\c__bnvs_crl_regex` Only used by `\__bnvs_prepare_crl:c`.

```
\regex_const:Nn \c__bnvs_crl_regex { \cO \{ | \cO \} }
```

(*End of definition for* `\c__bnvs_crl_regex`.)

`\__bnvs_prepare_crl:c` `\__bnvs_prepare_crl:c {⟨var⟩}`

Replace any literal {⟨...⟩} by `\BNVS_crl:n`{⟨...⟩}.

```
\BNVS_new:cpn { prepare_crl:c } #1 {
  \__bnvs_prepare:nnc { crl } { Unbalanced~\{~or~\} } { #1 }
}
```

`\c__bnvs_sqr_n_regex` Used by `\__bnvs_prepare_sqr:c` and `\__bnvs_resolve_prepare:nc`.

```
\regex_const:Nn \c__bnvs_sqr_n_regex { \[ ( [%\]
^\[]*) \] }
```

(*End of definition for* `\c__bnvs_sqr_n_regex`.)

`\c__bnvs_sqr_regex` Only used by `\__bnvs_prepare_sqr:c`.

```
\regex_const:Nn \c__bnvs_sqr_regex { \[|\] }
```

(*End of definition for* `\c__bnvs_sqr_regex`.)

`\__bnvs_prepare_sqr:c` `\__bnvs_prepare_sqr:c {⟨var⟩}`

Replace any literal `[⟨...⟩]` by `\BNVS_sqr:n{⟨...⟩}`.

```
\BNVS_new:cpn { prepare_sqr:c } #1 {
  \__bnvs_prepare:nnc { sqr } { Unbalanced~[~or~] } { #1 }
}
```

`\c__bnvs_rnd_n_regex` Only used by `\__bnvs_prepare_rnd:c`.

```
\regex_const:Nn \c__bnvs_rnd_n_regex { \( ( [%]
^\(]*) \) }
```

(*End of definition for* `\c__bnvs_rnd_n_regex`.)

`\c__bnvs_rnd_regex` Only used by `\__bnvs_prepare_rnd:c`.

```
\regex_const:Nn \c__bnvs_rnd_regex { \(|\) }
```

(*End of definition for* `\c__bnvs_rnd_regex`.)

`\__bnvs_prepare_rnd:c` `\__bnvs_prepare_rnd:c {⟨var⟩}`

Replace any literal `(⟨...⟩)` by `\BNVS_rnd:n{⟨...⟩}`.

```
\BNVS_new:cpn { prepare_rnd:c } #1 {
  \__bnvs_prepare:nnc { rnd } { Unbalanced~(~or~) } { #1 }
}
```

`\__bnvs_prepare:nc` `\__bnvs_prepare:nc {⟨definitions⟩} {⟨var⟩}`

Prepare the ⟨*definitions*⟩ into ⟨*var*⟩. Expand and convert the ⟨*definitions*⟩. Replace any literally material `⟨...⟩` enclosed between standard delimiters, by one of template `\BNVS_crl:n{⟨...⟩}`, `\BNVS_sqr:n{⟨...⟩}` or `\BNVS_rnd:n{⟨...⟩}`. After that, we are sure that commas are proper delimiters.

```
\BNVS_new:cpn { prepare:nc } #1 #2 {
  \__bnvs_convert:nc { #1 } { #2 }
  \__bnvs_prepare_crl:c   { #2 }
  \__bnvs_prepare_sqr:c  { #2 }
  \__bnvs_prepare_rnd:c   { #2 }
}
```

6.11.5 Other utilities

```
\BNVS_new:cpn { warn_until_s_stop:w } #1 \s_stop {
  \tl_if_empty:nF { #1 } {
    \BNVS_warning:n { Ignored:~#1 }
  }
}
\BNVS_new:cpn { scan_until_s_stop:w } {
  \peek_meaning:NTF \s_stop {
    \use_none:n
  } {
    \__bnvs_warn_until_s_stop:w
  }
}
```

__bnvs_peek_branch_until_s_stop:nnnnw `\__bnvs_peek_branch_until_s_stop:nnnnw {⟨sqr:n⟩} {⟨crl:n⟩} {⟨non empty:n⟩} {⟨empty:⟩} ⟨...⟩ \s_stop`

Branch according to the following token. Each argument, except ⟨*empty:*⟩, is the body of a function that takes one argument. It catches errors like `foo=[bar]baz`, where `baz` is not expected: no other argument should lay before `\s_stop`.

```
\cs_new:Npn \BNVS_sqr:n { \exp_not:N \BNVS_sqr:n }
\cs_new:Npn \BNVS_crl:n  { \exp_not:N \BNVS_crl:n  }
\BNVS_new:cpn { peek_branch_until_s_stop:nnnnw } #1 #2 #3 #4 {
  \peek_meaning:NTF \BNVS_sqr:n {
    \cs_set:Npn \BNVS_peek_branch:n ##1 { #1 }
    \cs_set:Npn \BNVS_peek_branch:nn ##1 ##2 {
      \BNVS_peek_branch:n { ##2 }
      \__bnvs_warn_until_s_stop:w
    }
    \BNVS_peek_branch:nn
  } {
    \peek_meaning:NTF \BNVS_crl:n {
      \cs_set:Npn \BNVS_peek_branch:n ##1 { #2 }
      \cs_set:Npn \BNVS_peek_branch:nn ##1 ##2 {
        \BNVS_peek_branch:n { ##2 }
        \__bnvs_warn_until_s_stop:w
      }
      \BNVS_peek_branch:nn
    } {
      \peek_meaning:NTF \s_stop {
        #4
        \use_none:n
      } {
        \cs_set:Npn \BNVS_peek_branch:w ##1 \s_stop {
          #3
        }
        \BNVS_peek_branch:w
      }
    }
  }
}
```

\c__bnvs_cB_regex Only used by `\__bnvs_keyval:ncc` and `\__bnvs_prepare_crl:c`.

```
\regex_const:Nn \c__bnvs_cB_regex { \cB . }
```

(*End of definition for* `\c__bnvs_cB_regex`.)

__bnvs_keyval:ncc `\__bnvs_keyval:ncc {⟨definitions⟩} {⟨V⟩} {⟨a⟩}`

Only called by `\__bnvs_keyval_ITND:w`. Parse ⟨*definitions*⟩ into the two `tl` variables with the given names (which should be different). Outer braces in ⟨*definitions*⟩ are kept. It is meant to be called within a group. On return ⟨*V*⟩ contains the leading standalone value, if any, ⟨a⟩ is a raw list of `\BNVS:nn{⟨key⟩}{⟨value⟩}` and `\BNVS:n{⟨key or value⟩}`.

```
\BNVS_new:cpn { keyval:ncc } #1 #2 #3 {
  \cs_set:Npn \BNVS: {
    \cs_set:Npn \BNVS:n ####1 {
      \__bnvs_tl_put_right:cn { #3 } { \BNVS:n { ####1 } }
    }
```

```
    \cs_set:Npn \BNVS:nn ####1 ####2 {
      \__bnvs_tl_put_right:cn { #3 } { \BNVS:nn { ####1 } { ####2 } }
    }
  }
  \cs_set:Npn \BNVS:n ##1 {
    \BNVS:
    \__bnvs_tl_set:cn { #2 } { ##1 }
  }
  \cs_set:Npn \BNVS:nn {
    \BNVS:
    \BNVS:nn
  }
  \__bnvs_tl_set:cn { #2 } { #1 }
  \BNVS_tl_use:nc {
    \regex_replace_all:NnN \c__bnvs_cB_regex { \c{BNVS_crl:} \0 }
  } { #2 }
  \__bnvs_tl_clear:c { #3 }
  \BNVS_tl_use:nv {
    \__bnvs_tl_clear:c { #2 }
    \keyval_parse:NNn \BNVS:n \BNVS:nn
  } { #2 }
  \BNVS_tl_use:nc {
    \regex_replace_all:nnN { \c{BNVS_crl:} } { }
  } { #2 }
  \BNVS_tl_use:nc {
    \regex_replace_all:nnN { \c{BNVS_crl:} } { }
  } { #3 }
}
```

Next function simply normalizes in place the content of the "N" `tl` variable.

```
\BNVS_new:cpn { normalize_N: } {
  \tl_if_empty:NF \l__bnvs_N_tl {
    \exp_args:NNe \tl_set:Nn \l__bnvs_N_tl {
      \exp_last_unbraced:NV \int_value:w \l__bnvs_N_tl \exp_stop_f:
    }
  }
}
```

6.11.6 Next index

```
\BNVS_int_new:c { next_index }
\BNVS_new:cpn { next_index:nn } #1 #2 {
  \__bnvs_int_incr:c { next_index }
  \exp_args:Nnne \__bnvs_is_gset:nnnT { #1 } { #2 } {
    = \int_use:N \l__bnvs_next_index_int
  } {
    \__bnvs_next_index:nn { #1 } { #2 }
  }
}
```

`\__bnvs_next_index:nnn` `\__bnvs_next_index:nnn {⟨id⟩} {⟨path⟩} {⟨code:n⟩}`

⟨*code:n*⟩ takes one argument: the next available index.

```
\BNVS_new:cpn { next_index:nnn } #1 #2 #3 {
  \BNVS_begin:
```

```
  \__bnvs_int_zero:c { next_index }
  \__bnvs_next_index:nn { #1 } { #2 }
  \cs_set:Npn \BNVS:n ##1 {
    \BNVS_end:
    \__bnvs_int_set:cn { next_index } { ##1 }
    #3
  }
  \BNVS_int_use:Nv \BNVS:n { next_index }
}
```

6.12 Definition

Lower level function names generally contain `_gdef`. We parse definitions and collect material between curly or square brackets. There is a pretreatment before the storage to anticipate the resolution.

`\__bnvs_gdef_by_key:nnn` `\__bnvs_gdef_by_key:(vvn|nvn)`

`\__bnvs_gdef_by_key:nnn {⟨id⟩} {⟨path⟩} {⟨definitions⟩}`

Called by itself through `\__bnvs_gdef_IPKD:w`. Here is the documentation. The typical example of indirect usage is `\Beanoves{⟨id⟩!⟨path⟩={⟨definitions⟩}}`. If ⟨*definitions*⟩ is empty, it clears out the data.

```
\BNVS_new:cpn { gdef_by_key:nnn } #1 #2 #3 {
  \tl_if_empty:nTF { #3 } {
    \__bnvs_gclear:nn { #1 } { #2 }
  } {
    \BNVS_begin:
    \keyval_parse:nnn {
      \__bnvs_ginit:nnn  { #1 } { #2 }
    } {
      \__bnvs_gdef_IPKD:w { #1 } { #2 }
    } { #3 }
    \BNVS_end:
  }
}
\BNVS_new:cpn { gdef_by_key:nvn } #1 {
  \BNVS_tl_use:nv { \__bnvs_gdef_by_key:nnn { #1 } }
}
\BNVS_new:cpn { gdef_by_key:vvn } {
  \BNVS_tl_use:Nvv \__bnvs_gdef_by_key:nnn
}
```

For ⟨*id*⟩!⟨*path*⟩=⟨*def*⟩ where ⟨*def*⟩ is the most general one. Workflow: `\__bnvs_gclear:nn` or

- `\__bnvs_keyval:ncc`
- `\__bnvs_gclear:nn` or `\__bnvs_gdef_by_index_keyval:nnn`
- `\__bnvs_gclear:nn` or `\__bnvs_gdef_by_key:nnn`
- `\__bnvs_ginit:nnn`.

`\__bnvs_keyval_ITND:w`

`\__bnvs_keyval_ITND:w {⟨id⟩} {⟨path⟩} {⟨index⟩} {⟨definitions⟩}`

Inly called by itself **(IN PROGRESS)**. Parses the definitions as a standalone value. Other values are errors. It does not mean that standalone values are free from errors by themselves.

```
\BNVS_new:cpn { keyval_ITND:w } #1 #2 #3 #4 {
  \tl_if_empty:nTF { #4 } {
    \__bnvs_gunset:nnn { #1 } { #2 } { =#3 }
    \__bnvs_gunset:nnn { #1 } { #2 } {  #3 }
  } {
    \BNVS_begin:
    \__bnvs_keyval:ncc { #4 } V a
    \BNVS_tl_last_unbraced:nv {
      \__bnvs_peek_branch_until_s_stop:nnnnw {
        \BNVS_error:n { No~`[...]`~allowed~with~index. }
      } {
```

For `ID!TAG.1={{FOO}}`, go recursive.

```
        \__bnvs_keyval_ITND:w { #1 } { #2 } { #3 } { ##1 }
      } {
        \__bnvs_ginit_ITND:w { #1 } { #2 } { #3 } { \BNVS:n { #4 } }
        \__bnvs_ginit:nnn { #1 } { #2.#3 } { #4 }
        \__bnvs_is_gset:nnnF { #1 } { #2 } 0 {
          \__bnvs_gset:nnne { #1 } { #2 } 0 {
            \exp_not:n { \BNVS_value:n { #4 } - } \int_eval:n { #3 - 1 }
          }
        }
      } {
      }
    } V \s_stop
    \__bnvs_tl_if_empty:cF a {
      \cs_set:Npn \BNVS:n  ##1 { \exp_not:n { ##1, } }
      \cs_set:Npn \BNVS:nn ##1 ##2 { \exp_not:n { ##1 = ##2, } }
      \BNVS_error:x { Ignored:~\__bnvs_tl_use:c a }
    }
    \BNVS_end:
  }
}
```

Support \BNVS_value:n

6.12.1 Preparation

6.12.2 Material between square brackets

Here is the documentation. The name of the function contains “square” or “by_index”.
Workflow:

- `\__bnvs_gdef_by_index:nnnn`
- `\__bnvs_gdef_by_index_keyval:nnn`
- `\__bnvs_gdef_by_index:nnn` or `\__bnvs_gdef_by_index:nnnn`
- `\__bnvs_gdef_by_index_keyval:nnn` or `\__bnvs_ginit:nnn`.
- `\__bnvs_if_set:ncccTF` `\__bnvs_gunset_deep:nn`.

Example: `\Beanoves{Id!Tag=[1=A,B,3=C]}`.

`\__bnvs_gdef_by_index:nnn` `\__bnvs_gdef_by_index:nnn {⟨id⟩} {⟨path⟩} {⟨definition⟩}`

To parse what is inside square brackets.
For:

- `\Beanoves{⟨id⟩!⟨path⟩=[⟨definitions⟩]}`,

- \Beanoves{⟨*id*⟩!⟨*path*⟩=[⟨*definition*⟩]} and
- \Beanoves{⟨*id*⟩!⟨*path*⟩=[⟨*index*⟩=⟨*definitions*⟩]}.

```
\BNVS_new:cpn { gdef_by_index:nnn } #1 #2 #3 {
```
Find the first available index.
```
  \__bnvs_next_index:nnn { #1 } { #2 } {
    \__bnvs_ginit_ITND:w { #1 } { #2 } { ##1 } { #3 }
  }
}
```

__bnvs_if_index:nTF __bnvs_if_index:nTF {⟨*what*⟩} {⟨*yes:n*⟩} {⟨*no:*⟩}

If ⟨*what*⟩ is an index, execute ⟨*yes:n*⟩ otherwise execute ⟨*no:*⟩. The ⟨*yes:n*⟩ argument is the body of a function with one argument: the normalized index built from ⟨*what*⟩.
```
\BNVS_new:cpn { if_index:nTF } #1 #2 #3 {
  \BNVS_begin:
  \__bnvs_match_if_once:NnTF \c__bnvs_A_index_Z_regex { #1 } {
    \__bnvs_match_pop:c N
    \__bnvs_normalize_N:
    \cs_set:Npn \BNVS:n ##1 {
      \BNVS_end:
      #2
    }
    \BNVS_tl_use:Nv \BNVS:n N
  } {
    \BNVS_end:
    #3
  }
}
```

__bnvs_gdef_by_index:nnnn __bnvs_gdef_by_index:nnnn {⟨*id*⟩} {⟨*path*⟩} {⟨*index*⟩} {⟨*definition*⟩}

To parse what is inside square brackets.
```
\BNVS_new:cpn { gdef_by_index:nnnn } #1 #2 #3 #4 {
  \__bnvs_if_index:nTF { #3 } {
    \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } { = ##1 } {
      \__bnvs_ginit_ITND:w { #1 } { #2 } { ##1 } { #4 }
    }
  } {
    \__bnvs_next_index:nnn { #1 } { #2 } {
      \__bnvs_ginit_ITND:w { #1 } { #2 } { ##1 } { #4 }
    }
  }
}
```

__bnvs_gdef_by_index_keyval:nnn __bnvs_gdef_by_index_keyval:nnn {⟨*id*⟩} {⟨*path*⟩} {⟨*definitions*⟩}

On execution, the `root tl` variable is set and not empty. Called by **(IN PROGRESS)** to parse what is inside square brackets:

- \Beanoves{⟨*path*⟩=[⟨*definitions*⟩]},
- \Beanoves{⟨*id*⟩!⟨*path*⟩=[⟨*definitions*⟩]}.

Calls __bnvs_gdef_by_index:nnn and __bnvs_gdef_by_index:nnnn.

```
\BNVS_new:cpn { gdef_by_index_keyval:nnn } #1 #2 #3 {
```
Remove what is related to tag.
```
  \tl_if_empty:nTF { #2 } {
    \BNVS_error:n { Unexpected~list~at~top~level. }
  } {
    \tl_if_empty:nF { #3 } {
      \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
        \__bnvs_gunset_deep:nn { #1 } { #2 }
        \BNVS_begin:
        \keyval_parse:nnn
          { \__bnvs_gdef_by_index:nnn  { #1 } { #2 } }
          { \__bnvs_gdef_by_index:nnnn { #1 } { #2 } }
          { #3 }
        \BNVS_end:
      }
    }
  }
}
\BNVS_new:cpn { gdef_by_index_keyval:nvn } #1 {
  \BNVS_tl_use:nv { \__bnvs_gdef_by_index_keyval:nnn { #1 } }
}
\BNVS_new:cpn { gdef_by_index_keyval:vvn } {
  \BNVS_tl_use:Nvv \__bnvs_gdef_by_index_keyval:nnn
}
\BNVS_new:cpn { gdef_by_index_keyval_I:nn } {
  \BNVS_tl_use:cv { gdef_by_index_keyval:nnn } I
}
\BNVS_new:cpn { gdef_by_index_keyval_I:vn } {
  \BNVS_tl_use:cv { gdef_by_index_keyval_I:nn }
}
```

6.12.3 Range or value lists

`\__bnvs_root_parsed:n` `\__bnvs_root_parsed:n {⟨key⟩}`

Called from `\__bnvs_root_keyval:nc`. Typically, for `\Beanoves{⟨key⟩}`. At the root level, the ⟨*key*⟩ is either ⟨*path*⟩ or ⟨*id*⟩`!`⟨*path*⟩, it is not a ⟨*subpath*⟩. Calls `\__bnvs_if_ITN:nTF` to parse the ⟨*key*⟩.

```
\BNVS_new:cpn { root_parsed:n } #1 {
  \__bnvs_if_ITN:nTF { #1 } {
    \__bnvs_gunset_deep:vv IT
    \__bnvs_tl_if_blank:vTF N {
      \__bnvs_ginit:vvn IT 1
    } {
      \BNVS_tl_use:Nvvv \__bnvs_ginit_ITND:w ITN 1
    }
  } {
    \BNVS_error:n { Unsupported~#1 }
  }
}
```

`\__bnvs_root_parsed:nn` `\__bnvs_root_parsed:nn {⟨key⟩} {⟨value⟩}`

Called from `\__bnvs_root_keyval:nc`. Typically, for

- `\Beanoves{⟨key⟩=⟨simple value⟩}`
- `\Beanoves{⟨key⟩=[⟨...⟩]}`
- `\Beanoves{⟨key⟩={⟨...⟩}}`

At the root level, the ⟨*key*⟩ is either ⟨*path*⟩ or ⟨*id*⟩`!`⟨*path*⟩, it is not a ⟨*subpath*⟩. Calls `\__bnvs_if_ITN:nTF` to parse the ⟨*key*⟩.

```
\BNVS_new:cpn { root_parsed:nn } #1 #2 {
  \__bnvs_if_ITN:nTF { #1 } {
    \__bnvs_tl_if_blank:vTF N {
      \__bnvs_gunset_deep:vv IT
      \__bnvs_peek_branch_until_s_stop:nnnnw {
```

For `\Beanoves{⟨key⟩=[⟨...⟩]}`, calls `\__bnvs_ginit_sqr:nnn`

```
        \__bnvs_ginit_sqr:vvn IT { #2 }
      } {
        \__bnvs_ginit_crl:vvn IT { #2 }
      } {
        \__bnvs_ginit:vvn IT { #2 }
      } { } #2 \s_stop
    } {
      \BNVS_tl_use:Nvvv \__bnvs_ginit_ITND:w ITN { #2 }
    }
  } {
    \BNVS_error:n { Unsupported~#1 }
  }
}
```

`\__bnvs_root_keyval:nc` `\__bnvs_root_keyval:nc {⟨definitions⟩} {⟨aux⟩}`

Is called by `\BeanovesReset` and by `\Beanoves` through `\__bnvs_root_keyval:NNn`, ⟨*definitions*⟩ directly comes from the user input. ⟨*aux*⟩ is an auxiliary variable only used within the body of the function, its content on input and output must be ignored whereas `J` may be updated.

1. Calls `\__bnvs_prepare:nc` to prepare ⟨*definitions*⟩ into the ⟨*aux*⟩ variable, which will contain only characters (with category code other).
2. Parse with keyval based on `\keyval_parsed:nn` helped by `\__bnvs_root_parsed:n` and `\__bnvs_root_parsed:nn`.

```
\BNVS_new:cpn { root_keyval:nc } #1 #2 {
  \__bnvs_prepare:nc { #1 } { #2 }
  \BNVS_tl_use:nv {
    \keyval_parse:NNn \__bnvs_root_parsed:n \__bnvs_root_parsed:nn
  } { #2 }
}
```

`\__bnvs_root_keyval:NNn` `\__bnvs_root_keyval:NNn ⟨is at top⟩ ⟨on provide mode⟩ {⟨definitions⟩}`

Called by `\Beanoves`. Calls `\__bnvs_root_keyval:nc` after some setup. The `a` tl auxiliary variable is used and, on return, the `J` variable may have been updated.

```
\BNVS_new:cpn { root_keyval:NNn } #1 #2 #3 {
  \bool_if:NT #1 {
```
At the document level, clear everything.
```
    \__bnvs_gclear:
  }
  \BNVS_begin:
  \bool_if:NTF #2 {
    \__bnvs_provide_ON:
  } {
    \__bnvs_provide_OFF:
  }
  \__bnvs_int_zero:c { i }
  \__bnvs_tl_set:cn a { #3 }
  \bool_if:NT #1 {
```
At the document level, also use the global definitions.
```
    \seq_if_empty:NF \g__bnvs_def_seq {
      \__bnvs_tl_put_left:cx a {
        \seq_use:Nn \g__bnvs_def_seq , ,
      }
    }
  }
  \BNVS_tl_use:Nv \__bnvs_root_keyval:nc a a
```
This is a list, possibly at the top
```
  \BNVS_end_tl_set:cv J J
}
```

\Beanoves `\Beanoves {`⟨*key-value list*⟩`}`

The keys are the slide overlay references. When no value is provided, it defaults to `1`. On the contrary, ⟨*key-value*⟩ items are parsed by `\__bnvs_root_keyval:NNn`.
```
\NewDocumentCommand \Beanoves { sm } {
  \__bnvs_set_false:c { reset }
  \__bnvs_set_false:c { reset_all }
  \__bnvs_set_false:c { only }
  \tl_if_empty:NTF \@currenvir {
```
In the preamble, record the definitions globally for later use. No expansion yet.
```
    \seq_gput_right:Nn \g__bnvs_def_seq { #2 }
  } {
    \tl_if_eq:NnTF \@currenvir { document } {
      \IfBooleanTF {#1} {
        \__bnvs_root_keyval:NNn \c_true_bool \c_true_bool
      } {
        \__bnvs_root_keyval:NNn \c_true_bool \c_false_bool
      }
    } {
      \IfBooleanTF {#1} {
        \__bnvs_root_keyval:NNn \c_false_bool \c_true_bool
      } {
        \__bnvs_root_keyval:NNn \c_false_bool \c_false_bool
      }
    } { #2 }
    \ignorespaces
  }
```

```
}
```
If we use the frame `beanoves` option, we can provide default values to the various overlay set names through thge use of `\Beanoves*`.
```
\define@key{beamerframe}{beanoves}{\Beanoves*{#1}}
```

6.13 Scanning named overlay specifications

Patch some beamer commands to support `?(...)` instructions in overlay specifications.

`\__bnvs_@frame` `\__bnvs_@frame {⟨overlay specification⟩}`
`\__bnvs_@masterdecode` `\__bnvs_@masterdecode {⟨overlay specification⟩}`

Preprocess ⟨*overlay specification*⟩ before beamer reads it. Both call `\__bnvs_resolve:nc`.

`\l__bnvs_ans_tl` Storage for the translated overlay specification, ⟨...⟩ instructions are replaced by their static counterparts.
(*End of definition for* `\l__bnvs_ans_tl`.)

Save the original macros `\beamer@frame` and `\beamer@masterdecode` then override them to properly preprocess the argument. We start by defining the overloads.

```
\makeatletter
\cs_set:Npn \__bnvs_@frame < #1 > {
  \BNVS_begin:
  \__bnvs_tl_clear:c { ans }
  \__bnvs_resolve:nc { #1 } { ans }
  \BNVS_set:cpn { :n } ##1 { \BNVS_end: \BNVS_saved@frame < ##1 > }
  \BNVS_tl_use:cv { :n } { ans }
}
\cs_set:Npn \__bnvs_@masterdecode #1 {
  \BNVS_begin:
  \__bnvs_tl_clear:c { ans }
  \__bnvs_resolve:nc { #1 } { ans }
  \BNVS_tl_use:nv {
    \BNVS_end:
    \BNVS_saved@masterdecode
  } { ans }
}
\cs_new:Npn \BeanovesOff {
  \cs_set_eq:NN \beamer@frame \BNVS_saved@frame
  \cs_set_eq:NN \beamer@masterdecode \BNVS_saved@masterdecode
}
\cs_new:Npn \BeanovesOn {
  \cs_set_eq:NN \beamer@frame \__bnvs_@frame
  \cs_set_eq:NN \beamer@masterdecode \__bnvs_@masterdecode
}
\AddToHook{begindocument/before}{
  \cs_if_exist:NTF \beamer@frame {
    \cs_set_eq:NN \BNVS_saved@frame \beamer@frame
    \cs_set_eq:NN \BNVS_saved@masterdecode \beamer@masterdecode
  } {
    \cs_set:Npn \BNVS_saved@frame < #1 > {
      \BNVS_error:n {Missing~package~beamer}
    }
    \cs_set:Npn \BNVS_saved@masterdecode < #1 > {
      \BNVS_error:n {Missing~package~beamer}
    }
```

```
  }
  \BeanovesOn
}
\makeatother
```

6.14 Resolution

Resolution means to transform `?(`⟨*queries*⟩`)` and alike into static beamer range expressions. This is a quite complex task because we allow at the same time algebraic expressions and beamer range expressions which seem like differences but are not algebraic expressions by themselves. Function names generally contain "resolve".

Each individual query is scanned multiple times, changing parts on each step to finally obtain only raw integers (along with management markers). Overall sketch: `\__bnvs_resolve:nc` is the top level resolution function. It is called by `\__bnvs_@frame`, `\__bnvs_@masterdecode` or `\BeanovesResolve` and for the resolution of embedded subqueries `?(`⟨*queries*⟩`)` or `!:(`⟨*queries*⟩`)` as well. The main functions used for resolution are

- `\__bnvs_resolve_query:nc`, called first, the input is a raw query, which is not a clist object, on return the answer is**(IN PROGRESS)**
  - `\BNVS_V:w{`⟨*v*⟩`}`, for a single integer literal value ⟨*v*⟩,
  - `\BNVS_AZ:w{`⟨a⟩`}{`⟨*z*⟩`}`, for a single range of integers from literal integer ⟨a⟩ to literal integer ⟨*z*⟩, or from literal integer ⟨a⟩ if ⟨*z*⟩ is empty,
  - a comma separated list of the above

  The result may be used as a single integer value as well as a beamer list of ranges. Depending on the context, the functions will be defined appropriately.

- `\__bnvs_resolve_prepare:nc`, called first by `\__bnvs_resolve_query:nc`, on return the prepared material consists of
  - `\BNVS_paren:n{`⟨*...*⟩`}` for material between rounded brackets,
  - `\BNVS_bIKDPTNUa:w{`⟨*b*⟩`}{`⟨*I*⟩`}{`⟨*K*⟩`}{`⟨*D*⟩`}{`⟨*P*⟩`}{`⟨*T*⟩`}{`⟨*N*⟩`}{`⟨*U*⟩`}{`⟨a⟩`}`, for specifications
  - `\BNVS_Ua:w{`⟨*U*⟩`}{`⟨a⟩`}` for static attributes after a "`.`" character and following `++`,
  - `\BNVS_n:n{`⟨*...*⟩`}` for an integer expression between square brackets,
  - `\BNVS_s:n{`⟨*...*⟩`}` for a general subquery inside `?(`⟨*...*⟩`)` ,
  - `\BNVS_assign:n{`⟨*|+|?*⟩`}`, for `=`, `+=` or `?=`,
  - `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
  - `\BNVS_comma:` for commas
  - `\BNVS_raw_to_stop:w`⟨*raw text*⟩`\s_stop` for everything else.

- `\__bnvs_resolve_subqueries:c`, after resolution of subqueries the material consists of
  - `\BNVS_bIPTNUa:w{`⟨*b*⟩`}{`⟨*I*⟩`}{`⟨*P*⟩`}{`⟨*T*⟩`}{`⟨*N*⟩`}{`⟨*U*⟩`}{`⟨a⟩`}`, for specifications
  - `\BNVS_Ua:w{.`⟨*attributes*⟩`}{`⟨*number of ++*⟩`}`,

- \BNVS_N:w{⟨...⟩} for a single static resolved integer,
- \BNVS_S:w{⟨...⟩} containing output similar to that of __bnvs_resolve_query:nc,
- \BNVS_assign:n{⟨|+|?⟩}, for =, += or ?=,
- \BNVS_two_colons: and \BNVS_colon:, range delimiters.
- \BNVS_comma: for commas
- \BNVS_raw:n{⟨raw text⟩} for everything else.

__bnvs_resolve_check:c after resolution the material consists of

- \BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}, for specifications
- \BNVS_N:w{⟨...⟩} for a single static resolved integer,
- \BNVS_S:w{⟨...⟩} for a comma separated list of static resolved ⟨A⟩, ⟨A⟩: and ⟨A⟩:⟨Z⟩ ranges, ⟨A⟩ and ⟨Z⟩ being unsigned integer decimal denotations,
- \BNVS_assign:n{⟨|+|?⟩}, for =, += or ?=,
- \BNVS_two_colons: and \BNVS_colon:, range delimiters.
- \BNVS_comma: for commas
- \BNVS_raw:n{⟨raw text⟩} for everything else.

__bnvs_resolve_rhs:c after resolution of assignments the material consists of **(IN PROGRESS)**

- \BNVS_V:w{⟨v⟩}, for a single integer literal value ⟨v⟩,
- \BNVS_AZ:w{⟨a⟩}{⟨z⟩}, for a single range of integers from literal integer ⟨a⟩ to literal integer ⟨z⟩, or from literal integer ⟨a⟩ if ⟨z⟩ is empty,
- a comma separated list of the above

__bnvs_resolve_assign:nc after resolution of assignments the material consists of

- \BNVS_bIPTNUa:w{⟨b⟩}{⟨I⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}, for specifications
- \BNVS_N:w{⟨...⟩} for a single static resolved integer,
- \BNVS_S:w{⟨...⟩} for a comma separated list of static resolved ⟨A⟩, ⟨A⟩: and ⟨A⟩:⟨Z⟩ ranges, ⟨A⟩ being positive and ⟨Z⟩ being nonnegative,
- \BNVS_two_colons: and \BNVS_colon:, range delimiters.
- \BNVS_comma: for commas
- \BNVS_raw:n{⟨raw text⟩} for everything else.

__bnvs_assign:Nwnnc

These functions define the function in the input and run the input.

The top level queries are a comma separated list of individual queries as augmented range specifications. Each individual query is temporarily stored in its own variable, then commands are inserted as markup. This variable is then executed to feed the top level resolution variable.

The augmentations in individual queries can take different forms. At the top level we have only ?(⟨queries⟩) or !:(⟨queries⟩) and inside we may have

- embedded `?(`⟨*queries*⟩`)` or `!:(`⟨*queries*⟩`)`,
- colon delimited ranges,
- various assignments (standard, with increment and as provider),
- references with possible square brackets specification,
- pre and post increment.

The right hand side expression in assignments may contain commas. In that case there can be a conflict with top level commas. To avoid such ambiguity, one can put rounded brackets around the critical part.

\c__bnvs_bIKDPN_regex Used by __bnvs_resolve_prepare:nc.

```
\regex_const:Nn \c__bnvs_bIKDPN_regex {
  ( (?: \+\+ )* )
  (?: ( \ur{c__bnvs_S_regex} )? (!) | (\.) )?
  ( \ur{c__bnvs_S_regex} (?: \. \ur{c__bnvs_S_regex} )* )
  (?: \. \s* ([-+\s]* \s* \d+ ) )?
  \s*
}
```

(*End of definition for* \c__bnvs_bIKDPN_regex.)

\c__bnvs_bIKDPTNUa_regex Used by __bnvs_resolve_prepare:nc.

⟨**b**⟩: an optional collection of `++` prefixes,

⟨**I**⟩: an optional ⟨*id*⟩

⟨**K**⟩: followed by a "`!`" or

⟨**D**⟩: an optional "`.`",

⟨**P**⟩: a dotted path,

⟨**T**⟩: optional dotted ⟨*attributes*⟩,

⟨**N**⟩: a litteral optionally signed integer,

⟨**U**⟩: other optional dotted ⟨*attributes*⟩,

⟨**a**⟩: an optional collection of `++` suffixes.

```
\regex_const:Nn \c__bnvs_bIKDPTNUa_regex {
  ( (?: \+\+ )* )
  (?: ( \ur{c__bnvs_S_regex} )? (!) | (\.) )?
  ( \ur{c__bnvs_S_regex} (?: \. \ur{c__bnvs_S_regex} )* )
  ( (?: \. \ur{c__bnvs_U_regex} )* )
  (?: \. \s* ([-+\s]* \s* \d+ ) )?
  ( (?: \. \ur{c__bnvs_U_regex} )* )
  ( (?: \+\+)* )
  \s*
}
```

(*End of definition for* \c__bnvs_bIKDPTNUa_regex.)

\c__bnvs_UaU_regex Used by __bnvs_resolve_prepare:nc.

⟨**U**⟩: optional dotted ⟨*attributes*⟩ and

⟨**a**⟩: post increment,

⟨*U*⟩**:** or standalone optional dotted ⟨*attributes*⟩.

```
\regex_const:Nn \c__bnvs_UaU_regex {
  (?: ( (?: \. \ur{c__bnvs_U_regex} )* ) ( (?: \+\+)+ )
    | ( (?: \. \ur{c__bnvs_U_regex} )+ ) )
  \s*
}
```

(*End of definition for* `\c__bnvs_UaU_regex`.)

`\__bnvs_resolve_prepare:nc` `\__bnvs_resolve_prepare:nc {⟨queries⟩} {⟨var⟩}`

Called by `\__bnvs_resolve_query:nc` and by `\__bnvs_resolve_queries:nc`. Prepare the ⟨*queries*⟩ into ⟨*var*⟩. Inside ⟨*queries*⟩, ranges are denoted with a colon syntax.

On return, every material has been tagged between commands, as a function or a command argument. More precisely ⟨*var*⟩ consists of

- `\BNVS_paren:n{⟨...⟩}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{⟨b⟩}{⟨I⟩}{⟨K⟩}{⟨D⟩}{⟨P⟩}{⟨T⟩}{⟨N⟩}{⟨U⟩}{⟨a⟩}`, for specifications
- `\BNVS_Ua:w{⟨U⟩}{⟨a⟩}` for static attributes after a "`.`" character and following `++`,
- `\BNVS_n:n{⟨...⟩}` for an integer expression between square brackets,
- `\BNVS_s:n{⟨...⟩}` for a general subquery inside `?(⟨...⟩)` ,
- `\BNVS_assign:n{⟨|+|?⟩}`, for `=`, `+=` or `?=`,
- `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
- `\BNVS_comma:` for commas
- `\BNVS_raw_to_stop:w⟨raw text⟩\s_stop` for everything else.

```
\BNVS_new:cpn { resolve_prepare:nc } #1 #2 {
```

One shot function, subsequent calls within the same group are just restricted to the raw assignement of ⟨*queries*⟩ to ⟨*var*⟩, because the ⟨*queries*⟩ argument here is some part of a more general ⟨*queries*⟩ that has already been prepared.

```
  \BNVS_set:cpn { resolve_prepare:nc } ##1 ##2 {
    \__bnvs_tl_set:cn { ##2 } { ##1 }
  }
```

Turn the queries into a string by expanding macros, `#2` will contain only characters.

```
  \tl_if_empty:nTF { #1 } {
    \__bnvs_tl_clear:c { #2 }
  } {
    \__bnvs_tl_set:ce { #2 } { \exp_args:Nc \cs_to_str:N { #1 } }
  }
```

Enclose the material between commas, there is no grouping yet so this is not a comma separated list. The leading and trailing marks will be inserted at the end of this function body.

```
  \BNVS_tl_use:Nc \tl_replace_all:Nnn { #2 }
    { , } { \s_stop \BNVS_comma: \BNVS_raw_to_stop:w}
```

- Replace “(⟨...⟩)” with
  \s_stop
  \BNVS_paren:n{\BNVS_raw_to_stop:w⟨...⟩\s_stop}
  \BNVS_raw_to_stop:w.
  Standard delimiters are expected to be properly balanced, no error recovery policy for that matter. Category codes now come back into play.

```
\cs_set:Npn \BNVS_loop: {
  \BNVS_tl_use:nc { \regex_replace_all:NnNT \c__bnvs_rnd_n_regex {
    \c { s_stop }
    \c { BNVS_paren:n } \cB \{
      \c { BNVS_raw_to_stop:w } \1 \c { s_stop }
    \cE \}
    \c { BNVS_raw_to_stop:w }
  } } { #2 } \BNVS_loop:
}
\BNVS_loop:
```

- Replace “[⟨...⟩]” with
  \s_stop
  \BNVS_v:n{\BNVS_raw_to_stop:w⟨...⟩\s_stop}
  \BNVS_raw_to_stop:w.
  The content will be resolved as a static signed integer and will be used as an index.

```
\cs_set:Npn \BNVS_loop: {
  \BNVS_tl_use:nc { \regex_replace_all:NnNT \c__bnvs_sqr_n_regex {
      \c { s_stop } \c { BNVS_n:n } \cB \{
        \c { BNVS_raw_to_stop:w } \1 \c { s_stop }
      \cE \} \c { BNVS_raw_to_stop:w }
  } } { #2 } \BNVS_loop:
}
\BNVS_loop:
```

- Also replace references, increments, assignments and colons.

```
\BNVS_tl_use:nc { \regex_replace_case_all:nN {
```

  Basically, “?(⟨...⟩)” is replaced by “\BNVS_s:n{⟨...⟩}” that will be evaluated as a subquery for a clist of values and ranges.

```
  { (?: \?|!: ) \c { s_stop } \c { BNVS_paren:n } } {
                \c { s_stop } \c { BNVS_s:n     }
  }
  { \c__bnvs_bIKDPTNUa_regex } {
    \c { s_stop }
    \c { BNVS_bIKDPTNUa:w }
       { \1 } { \2 } { \3 } { \4 } { \5 } { \6 } { \7 } { \8 } { \9 }
    \c { BNVS_raw_to_stop:w }
  }
```

  Post increment operators and attributes are collected

```
  { \c__bnvs_UaU_regex } {
    \c { s_stop }
    \c { BNVS_Ua:w } \cB \{ \1\3 \cE \} \cB \{ \2 \cE \}
    \c { BNVS_raw_to_stop:w }
  }
```

  Assignment operators together with their optional modifier are collected

```
    { \s* ([+?])?= \s*  } {
      \c { s_stop }
      \c { BNVS_assign:n } \cB \{ \1 \cE \}
      \c { BNVS_raw_to_stop:w }
    }
```

Colon range delimiters:

```
    { \s* ::+ \s* } {
      \c { s_stop }
      \c { BNVS_two_colons: }
      \c { BNVS_raw_to_stop:w }
    }
    { \s* : \s* } {
      \c { s_stop }
      \c { BNVS_colon: }
      \c { BNVS_raw_to_stop:w }
    }
  } } { #2 }
  \__bnvs_tl_put_left:cn  { #2 } { \BNVS_raw_to_stop:w }
  \__bnvs_tl_put_right:cn { #2 } { \s_stop }
}
```

`\__bnvs_resolve_queries:nc` `\__bnvs_resolve_queries:nc {⟨queries⟩} {⟨ans⟩}`

One of the top level resolution entry point. May be called recursively. For each individual ⟨*query*⟩ of the ⟨*queries*⟩ `clist`, it replaces in the ⟨*queries*⟩ all the `?(⟨...⟩)` query instructions as well as `[⟨...⟩]` index references with their static counterpart then it evaluates the instructions to obtain a single value or a set of references. Ranges are in colon (`:`) syntax. The function `\__bnvs_resolve_query:nc` is called for each individual query in the ⟨*queries*⟩ `clist`. The `Q seq` variable stores the specifications whereas the `Q tl` variable stores the result. The implementation is not highly efficient in the sense that queries are scanned multiple times, but it does not cost that much as long as queries are reasonably short.

```
\BNVS_new:cpn { resolve_queries:nc } #1 #2 {
  \BNVS_begin:
  \__bnvs_tl_clear:c  Q
  \__bnvs_seq_clear:c Q
  \cs_set:Npn \BNVS_raw:n ##1 {
    \tl_if_blank:nF { ##1 } {
      \__bnvs_seq_put_right:cn Q { ##1 }
    }
  }
  \cs_set:Npn \BNVS_set_S:n ##1 {
    \__bnvs_tl_if_empty:cT Q {
      \__bnvs_tl_set:cn  Q { \BNVS_S:w }
      \cs_set:Npn \BNVS_S:w ####1 {
        \__bnvs_seq_put_right:cn Q { ####1 }
      }
      \cs_set_eq:NN \BNVS_V:w \BNVS_S:w
    }
    \__bnvs_seq_put_right:cn Q { ##1 }
  }
  \cs_set_eq:NN \BNVS_S:w \BNVS_set_S:n
```

```
    \cs_set:Npn \BNVS_V:w ##1 {
      \__bnvs_seq_put_right:cn Q { ##1 }
      \cs_set_eq:NN \BNVS_V:w \BNVS_set_S:n
    }
    \clist_map_inline:nn { #1 } {
      \__bnvs_resolve_query:nc { ##1 } { #2 }
      \BNVS_tl_use:Nv \use:n { #2 }
    }
    \__bnvs_tl_if_empty:cT Q {
      \__bnvs_tl_set:cn  Q { \BNVS_V:w }
    }
    \exp_args:Nne \__bnvs_tl_put_right:cn Q {
      { { \__bnvs_seq_use:cn Q { , } } }
    }
    \BNVS_end_tl_set:cv { #2 } Q
}
```

`\__bnvs_resolve_argument:c` `\__bnvs_resolve_argument:c {`⟨*query var*⟩`}`
`\__bnvs_resolve_argument:nc` `\__bnvs_resolve_argument:nc {`⟨*argument*⟩`} {`⟨*var*⟩`}`

Called by `\__bnvs_resolve_subqueries:nc`.
On input, ⟨*query*⟩ or ⟨*var*⟩ consists of

- `\BNVS_paren:n{`⟨...⟩`}` for material between rounded brackets,
- `\BNVS_bIKDPTNUa:w{`⟨*b*⟩`}{`⟨*I*⟩`}{`⟨*K*⟩`}{`⟨*D*⟩`}{`⟨*P*⟩`}{`⟨*T*⟩`}{`⟨*N*⟩`}{`⟨*U*⟩`}{`⟨a⟩`}`, for specifications
- `\BNVS_Ua:w{`⟨*U*⟩`}{`⟨a⟩`}` for static attributes after a "`.`" character and following `++`,
- `\BNVS_n:n{`⟨...⟩`}` for an integer expression between square brackets,
- `\BNVS_s:n{`⟨...⟩`}` for a general subquery inside `?(`⟨...⟩`)` ,
- `\BNVS_assign:n{`⟨*|+|?*⟩`}`, for `=`, `+=` or `?=`,
- `\BNVS_two_colons:` and `\BNVS_colon:` are range delimiters
- `\BNVS_comma:` for commas
- `\BNVS_raw_to_stop:w`⟨*raw text*⟩`\s_stop` for everything else.

and on output, ⟨*var*⟩ consists of

- `\BNVS_bIPTNUa:w{`⟨*b*⟩`}{`⟨*I*⟩`}{`⟨*P*⟩`}{`⟨*T*⟩`}{`⟨*N*⟩`}{`⟨*U*⟩`}{`⟨a⟩`}`, for specifications
- `\BNVS_N:w{`⟨...⟩`}` for a single static resolved integer,
- `\BNVS_S:w{`⟨...⟩`}` for a comma separated list of static resolved ⟨*A*⟩, ⟨*A*⟩: and ⟨*A*⟩:⟨*Z*⟩ ranges, ⟨*A*⟩ being positive and ⟨*Z*⟩ being nonnegative,
- `\BNVS_two_colons:` and `\BNVS_colon:`, range delimiters.
- `\BNVS_comma:` for commas
- `\BNVS_raw:n{`⟨*raw text*⟩`}` for everything else.

The ⟨*query var*⟩ is executed after the commands it contains are defined on the run. use __bnvs_resolve_query:nc. Run inside a group.

```
\BNVS_new:cpn { resolve_argument:nc } #1 #2 {
  \BNVS_begin:
  \__bnvs_resolve_subqueries:nc { ##1 } A
  \__bnvs_resolve_check:c A
  \__bnvs_resolve_assign:c A
  \BNVS_end:
}
```

Raw text.

__bnvs_resolve_subqueries:c __bnvs_resolve_subqueries:c {⟨*query var*⟩}
__bnvs_resolve_subqueries:nc __bnvs_resolve_subqueries:nc {⟨*query*⟩} {⟨*var*⟩}

Called after __bnvs_resolve_prepare:nc.

On input, ⟨*query*⟩ or ⟨*var*⟩ consists of

- \BNVS_paren:n{⟨...⟩} for material between rounded brackets,
- \BNVS_bIKDPTNUa:w{⟨*b*⟩}{⟨*I*⟩}{⟨*K*⟩}{⟨*D*⟩}{⟨*P*⟩}{⟨*T*⟩}{⟨*N*⟩}{⟨*U*⟩}{⟨a⟩}, for specifications
- \BNVS_Ua:w{⟨*U*⟩}{⟨a⟩} for static attributes after a "." character and following ++,
- \BNVS_n:n{⟨...⟩} for an integer expression between square brackets,
- \BNVS_s:n{⟨...⟩} for a general subquery inside ?(⟨...⟩) ,
- \BNVS_assign:n{⟨*|+|?*⟩}, for =, += or ?=,
- \BNVS_two_colons: and \BNVS_colon: are range delimiters
- \BNVS_comma: for commas
- \BNVS_raw_to_stop:w⟨*raw text*⟩\s_stop for everything else.

and on output, ⟨*var*⟩ consists of

- \BNVS_bIPTNUa:w{⟨*b*⟩}{⟨*I*⟩}{⟨*P*⟩}{⟨*T*⟩}{⟨*N*⟩}{⟨*U*⟩}{⟨a⟩}, for specifications
- \BNVS_Ua:w{.⟨*attributes*⟩}{⟨*number of ++*⟩},
- \BNVS_N:w{⟨...⟩} for a single static resolved integer,
- \BNVS_S:w{⟨...⟩} containing output similar to that of __bnvs_resolve_query:nc,
- \BNVS_assign:n{⟨*|+|?*⟩}, for =, += or ?=,
- \BNVS_two_colons: and \BNVS_colon:, range delimiters.
- \BNVS_comma: for commas
- \BNVS_raw:n{⟨*raw text*⟩} for everything else.

The ⟨*query var*⟩ is executed after the commands it contains are defined on the run. In particular, \BNVS_n:n and \BNVS_s:n use __bnvs_resolve_query:nc. Run inside a group.

```
\BNVS_new:cpn { resolve_subqueries:nc } #1 #2 {
```